

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO
CÁC HỆ THỐNG PHÂN TÁN
ĐỀ TÀI PHÁT TRIỂN ỨNG DỤNG MICROSERVICE

Giảng viên: TS. Kim Ngọc Bách

Nhóm học phần: 02

Danh sách thành viên - Nhóm: 07

Họ và tên	Mã sinh viên
Nguyễn Thạc Bảo	B22DCCN067
Nguyễn Thị Huế	B22DCCN355
Nguyễn Tiến Hưng	B22DCCN415

Hà Nội, 2026

LỜI CẢM ƠN

Trong quá trình thực hiện báo cáo môn học Các hệ thống phân tán với đề tài “*Phát triển ứng dụng Microservice*”, nhóm chúng em đã có cơ hội tìm hiểu thêm nhiều kiến thức liên quan đến kiến trúc Microservice, RESTful API, Message Queue cũng như cách xây dựng và triển khai hệ thống phân tán trong thực tế.

Nhóm xin gửi lời cảm ơn đến TS. Kim Ngọc Bách – giảng viên bộ môn đã giảng dạy và truyền đạt cho chúng em những kiến thức nền tảng của môn học trong suốt quá trình học tập. Những nội dung về hệ thống phân tán, giao tiếp giữa các service và kiến trúc phần mềm hiện đại đã giúp nhóm có định hướng để thực hiện đề tài này.

Chúng em cũng xin gửi lời cảm ơn đến các thành viên trong nhóm, những người đã cùng nhau hợp tác, đóng góp ý tưởng và nỗ lực hoàn thành bài báo cáo. Trong quá trình thực hiện đề tài, mỗi thành viên đều đảm nhận những phần công việc khác nhau và cùng hỗ trợ nhau để hoàn thiện nội dung cũng như các chức năng của hệ thống. Sự phối hợp và tinh thần làm việc nhóm là yếu tố quan trọng giúp nhóm hoàn thành đề tài đúng tiến độ.

Bên cạnh đó, quá trình thực hiện báo cáo cũng giúp nhóm rèn luyện thêm kỹ năng tìm hiểu tài liệu, làm việc nhóm và áp dụng kiến thức đã học vào thực tế. Tuy vẫn còn nhiều hạn chế về kinh nghiệm và thời gian thực hiện, nhưng đây là cơ hội để nhóm hiểu rõ hơn về mô hình Microservice và các công nghệ được sử dụng trong hệ thống phân tán hiện đại.

Do thời gian thực hiện còn hạn chế và kiến thức thực tế chưa nhiều nên báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến nhận xét và góp ý để có thể hoàn thiện hơn trong các bài báo cáo tiếp theo.

Nhóm xin chân thành cảm ơn!

MỤC LỤC

DANH MỤC BẢNG BIỂU	I
DANH MỤC HÌNH ẢNH	II
BẢNG THUẬT NGỮ VIẾT TẮT	III
MỞ ĐẦU	1
1. Lý do chọn đề tài	1
2. Mục tiêu đề tài	1
3. Phạm vi đề tài	2
4. Phương pháp thực hiện	3
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT.....	5
1.1. Tổng quan về kiến trúc Microservice	5
1.1.1. Khái niệm Microservice.....	6
1.1.2. Đặc điểm của Microservice	6
1.1.3. Ưu điểm và nhược điểm.....	7
1.1.4. So sánh Monolithic và Microservice.....	7
1.2. RESTful API	8
1.2.1. Khái niệm REST	8
1.2.2. Nguyên tắc thiết kế RESTful API	9
1.2.3. HTTP Methods.....	9
1.2.4. HTTP Status Code	10
1.3. Message Queue	10
1.3.1. Khái niệm Message Queue	11
1.3.2. Mô hình Producer – Consumer.....	12
1.3.3. RabbitMQ / Kafka / Redis Stream	12
1.3.4. Giao tiếp bất đồng bộ trong microservice	13
1.4. Docker và Docker Compose	13
1.4.1. Khái niệm container	14
1.4.2. Docker Compose	14

1.4.3. Triển khai nhiều service	15
1.5. JWT Authentication.....	15
1.5.1. Khái niệm JWT	15
1.5.2. Quy trình xác thực	16
1.5.3. Bảo mật API.....	16
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	18
2.1. Mô tả bài toán	18
2.2. Yêu cầu hệ thống	19
2.2.1. Yêu cầu chức năng	19
2.2.2. Yêu cầu phi chức năng.....	20
2.3. Kiến trúc tổng thể hệ thống	21
2.3.1. Sơ đồ kiến trúc microservice	21
2.3.2. Luồng xử lý nghiệp vụ.....	22
2.4. Thiết kế các Microservice	23
2.4.1. Thiết kế API Gateway.....	23
2.4.2. Thiết kế Service Registry	24
2.4.3. Thiết kế User Service	25
2.4.4. Thiết kế Product Service.....	26
2.4.5. Thiết kế Order Service	26
2.4.6. Thiết kế Payment Service.....	27
2.4.7. Thiết kế Recommendation Service.....	28
2.4.8. Thiết kế Notification Service	28
2.4.9. Thiết kế giao tiếp giữa các Microservice	28
2.4.10. Thiết kế cơ sở dữ liệu phân tán	29
2.4.11. Thiết kế triển khai hệ thống bằng Docker	29
2.5. Thiết kế cơ sở dữ liệu	30
2.5.1. Thiết kế cơ sở dữ liệu User Service.....	31
2.5.2. Thiết kế cơ sở dữ liệu Product Service.....	31

2.5.3. Thiết kế cơ sở dữ liệu <i>Order Service</i>	32
2.5.4. Thiết kế cơ sở dữ liệu <i>Payment Service</i>	32
2.5.5. Thiết kế cơ sở dữ liệu <i>Recommendation Service</i>	33
2.5.6. Thiết kế phân tán dữ liệu giữa các service	33
2.6. Thiết kế RESTful API.....	34
2.6.1. Tổng quan thiết kế <i>RESTful API</i>	34
2.6.2. Phân loại API theo cơ chế xác thực.....	34
2.6.3. Thiết kế API của <i>User Service</i>	35
2.6.4. Thiết kế API của <i>Product Service</i>	36
2.6.5. Thiết kế API của <i>Order Service</i>	36
2.6.6. Thiết kế API của <i>Payment Service</i>	37
2.6.7. Thiết kế API của <i>Recommendation Service</i>	37
2.6.8. Thiết kế chuẩn <i>Request</i> và <i>Response</i>	37
2.7. Thiết kế Message Queue	38
2.7.1. Các queue sử dụng.....	39
2.7.2. <i>Producer</i> và <i>Consumer</i>	40
2.7.3. Luồng message.....	41
2.8. Thiết kế xử lý lỗi	42
CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG	45
3.1. Công nghệ sử dụng	45
3.2. Xây dựng các service	45
3.3. Xây dựng giao tiếp RESTful API.....	49
3.4. Xây dựng Message Queue.....	51
3.5. Triển khai Docker Compose.....	52
3.6. Logging và Monitoring	54
CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ	58
4.1. Môi trường kiểm thử.....	58
4.2. Kiểm thử chức năng	59

4.3. Kiểm thử lỗi.....	62
4.4. Đánh giá hệ thống.....	63
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	65
1. Kết quả đạt được	65
2. Hạn chế và các vấn đề còn tồn tại.....	65
3. Hướng phát triển.....	66
TÀI LIỆU THAM KHẢO.....	67

DANH MỤC BẢNG BIỂU

Bảng 1.1. Ưu điểm của kiến trúc Microservice.....	7
Bảng 1.2. Nhược điểm của kiến trúc Microservice.....	7
Bảng 1.3. Các HTTP Method phổ biến.	9
Bảng 1.4. Các nhóm HTTP Status Code phổ biến.	10
Bảng 2.1. Thiết kế các queue sử dụng.....	39
Bảng 3.1. Các service và vai trò của nó trong hệ thống.	46
Bảng 3.2. Các dạng giao tiếp REST chính.	50
Bảng 3.3. Các tệp Docker Compose và vai trò trong hệ thống.	53

DANH MỤC HÌNH ẢNH

Hình 1.1. Mô hình kiến trúc Microservice.	5
Hình 2.1. Kiến trúc tổng quan hệ thống.	21
Hình 3.1. Các Docker container của các service trong hệ thống.....	45
Hình 3.2. Các service được register vào Eureka để giao tiếp với nhau.....	47
Hình 3.3. Các queue của hệ thống.....	51
Hình 3.4. Monitoring của server.....	55
Hình 3.5. Logging hệ thống qua Loki + Promtail (1).....	55
Hình 3.6. Logging hệ thống qua Loki + Promtail (2).....	56
Hình 3.7. Distribute tracing của request tới server.....	56
Hình 3.8. Các cấu hình gửi cảnh báo đến admin.....	57
Hình 3.9. Cảnh báo được gửi qua telegram.....	57

BẢNG THUẬT NGỮ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Ý nghĩa
AMQP	Advanced Message Queuing Protocol	Giao thức truyền thông cho hệ thống Message Queue
API	Application Programming Interface	Giao diện lập trình ứng dụng
CMS	Content Management System	Hệ thống quản lý nội dung
DB	Database	Cơ sở dữ liệu
Docker	Docker Platform	Nền tảng container hóa ứng dụng
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HyperText Transfer Protocol Secure	Giao thức HTTP bảo mật
IP	Internet Protocol	Giao thức địa chỉ mạng
JSON	JavaScript Object Notation	Định dạng dữ liệu trao đổi
JWT	JSON Web Token	Chuẩn token xác thực người dùng
Kafka	Apache Kafka	Nền tảng xử lý luồng dữ liệu phân tán

Kubernetes	Kubernetes Container Orchestration	Nền tảng điều phối container
MySQL	My Structured Query Language	Hệ quản trị cơ sở dữ liệu quan hệ
REST	Representational State Transfer	Kiến trúc thiết kế dịch vụ web
RESTful API	RESTful Application Programming Interface	API được xây dựng theo kiến trúc REST
Redis	Remote Dictionary Server	Hệ quản trị dữ liệu dạng key-value dùng để cache
RabbitMQ	Rabbit Message Queue	Hệ thống hàng đợi thông điệp
SMS	Short Message Service	Dịch vụ tin nhắn văn bản
SQL	Structured Query Language	Ngôn ngữ truy vấn cơ sở dữ liệu
URI	Uniform Resource Identifier	Định danh tài nguyên trên web
UI	User Interface	Giao diện người dùng
Vite	Vite Build Tool	Công cụ build frontend tốc độ cao
YAML	YAML Ain't Markup Language	Định dạng file cấu hình

MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, kiến trúc Microservice đã trở thành xu hướng phổ biến trong quá trình phát triển các hệ thống phần mềm hiện đại. Khác với mô hình nguyên khối (Monolithic Architecture), kiến trúc Microservice cho phép chia hệ thống thành nhiều dịch vụ nhỏ, độc lập và có thể triển khai riêng biệt. Cách tiếp cận này giúp tăng khả năng mở rộng, dễ bảo trì, nâng cao tính linh hoạt và phù hợp với các hệ thống có quy mô lớn hoặc lượng người dùng cao.

Bên cạnh đó, sự phát triển mạnh mẽ của điện toán đám mây, container hóa và các công nghệ giao tiếp liên dịch vụ như RESTful API và Message Queue đã tạo điều kiện thuận lợi cho việc triển khai mô hình Microservice trong thực tế. Nhiều doanh nghiệp và nền tảng lớn hiện nay như Netflix, Amazon, Shopee hay Grab đều áp dụng kiến trúc này nhằm nâng cao hiệu năng và khả năng mở rộng hệ thống.

Xuất phát từ nhu cầu thực tiễn đó, nhóm lựa chọn đề tài “Xây dựng hệ thống thương mại điện tử bán đồ thể thao theo kiến trúc Microservice sử dụng RESTful API và Message Queue” nhằm nghiên cứu và xây dựng một hệ thống phần mềm theo hướng dịch vụ phân tán. Đề tài không chỉ giúp nhóm tiếp cận mô hình phát triển phần mềm hiện đại mà còn tạo điều kiện vận dụng các kiến thức về lập trình backend, cơ sở dữ liệu, giao tiếp liên dịch vụ, xử lý bất đồng bộ và triển khai hệ thống.

Ngoài ra, việc thực hiện đề tài còn giúp nhóm hiểu rõ hơn về các vấn đề trong hệ phân tán như tính nhất quán dữ liệu, xử lý lỗi giữa các service, giao tiếp đồng bộ và bất đồng bộ, cũng như khả năng giám sát và quản lý hệ thống trong môi trường thực tế.

2. Mục tiêu đề tài

Đề tài hướng đến mục tiêu xây dựng một hệ thống thương mại điện tử bán đồ thể thao theo kiến trúc Microservice với các dịch vụ hoạt động độc lập và có khả năng giao tiếp với nhau thông qua RESTful API và Message Queue.

Các mục tiêu cụ thể gồm:

- Xây dựng hệ thống gồm nhiều microservice độc lập, mỗi service phụ trách một chức năng riêng biệt.

- Thiết kế và triển khai RESTful API cho các service nhằm phục vụ giao tiếp đồng bộ giữa client và các service trong hệ thống.
- Ứng dụng Message Queue để thực hiện giao tiếp bất đồng bộ giữa các service trong các luồng nghiệp vụ.
- Tách biệt dữ liệu giữa các service nhằm đảm bảo tính độc lập và giảm sự phụ thuộc lẫn nhau.
- Xây dựng cơ chế xử lý lỗi cơ bản như logging, retry và kiểm tra trạng thái service.
- Triển khai hệ thống bằng Docker/Docker Compose để các service có thể hoạt động độc lập.
- Thực hiện kiểm thử các luồng nghiệp vụ liên service và đánh giá khả năng hoạt động của hệ thống.
- Tìm hiểu và áp dụng các công nghệ hiện đại trong phát triển hệ thống phân tán như API Gateway, RabbitMQ, Redis, Monitoring và Logging.

Thông qua đề tài, nhóm mong muốn nâng cao kỹ năng thiết kế hệ thống, phát triển backend và triển khai ứng dụng theo mô hình phần mềm hiện đại.

3. Phạm vi đề tài

Đề tài tập trung xây dựng một hệ thống thương mại điện tử bán đồ thể thao theo kiến trúc Microservice nhằm mô phỏng mô hình hoạt động của các nền tảng bán hàng trực tuyến hiện đại. Hệ thống được phát triển theo hướng phân tán, trong đó các chức năng nghiệp vụ được tách thành nhiều service độc lập và có khả năng triển khai riêng biệt. Các service chính của hệ thống bao gồm User Service dùng để quản lý người dùng và xác thực tài khoản, Product Service quản lý thông tin sản phẩm thể thao, Order Service xử lý đơn hàng, Payment Service thực hiện thanh toán và Notification Service đảm nhận việc gửi thông báo cho khách hàng.

Hệ thống hỗ trợ các chức năng cơ bản của một website thương mại điện tử như đăng ký và đăng nhập tài khoản, quản lý thông tin sản phẩm, tìm kiếm sản phẩm, tạo đơn hàng, xử lý thanh toán và gửi thông báo sau khi đặt hàng thành công. Các service giao tiếp với nhau thông qua hai hình thức chính là RESTful API cho các tác vụ đồng

bộ và Message Queue cho các tác vụ bất đồng bộ nhằm đảm bảo khả năng mở rộng và giảm sự phụ thuộc giữa các service.

Trong đề tài, mỗi service được tổ chức với vùng dữ liệu riêng nhằm đảm bảo tính độc lập và phù hợp với nguyên tắc của kiến trúc Microservice. Hệ thống sử dụng các công nghệ như MySQL hoặc PostgreSQL để lưu trữ dữ liệu nghiệp vụ, Redis để cache dữ liệu và RabbitMQ để xử lý hàng đợi thông điệp. Ngoài ra, đề tài còn triển khai Docker và Docker Compose để hỗ trợ việc triển khai và vận hành các service một cách độc lập.

Bên cạnh các chức năng chính, hệ thống còn tích hợp cơ chế logging và monitoring cơ bản nhằm theo dõi hoạt động của các service cũng như hỗ trợ xử lý lỗi trong quá trình giao tiếp giữa các thành phần của hệ thống. Tuy nhiên, do giới hạn về thời gian và nguồn lực, đề tài chưa tập trung triển khai các kỹ thuật nâng cao như tối ưu hiệu năng ở quy mô lớn, triển khai Kubernetes hoàn chỉnh, bảo mật chuyên sâu hay xử lý distributed transaction phức tạp trong môi trường production.

4. Phương pháp thực hiện

Để thực hiện đề tài, nhóm tiến hành nghiên cứu và triển khai theo nhiều giai đoạn khác nhau nhằm đảm bảo hệ thống được xây dựng đúng theo định hướng của kiến trúc Microservice. Trước tiên, nhóm tập trung nghiên cứu các kiến thức nền tảng liên quan đến hệ thống phân tán, kiến trúc Microservice, RESTful API, Message Queue và cơ chế giao tiếp giữa các service. Đồng thời, nhóm cũng tìm hiểu các công nghệ hỗ trợ triển khai hệ thống như Docker, RabbitMQ, Redis và các công cụ quản lý cơ sở dữ liệu nhằm lựa chọn giải pháp phù hợp cho bài toán thương mại điện tử.

Sau giai đoạn nghiên cứu lý thuyết, nhóm tiến hành phân tích yêu cầu bài toán và thiết kế kiến trúc tổng thể cho hệ thống. Dựa trên đặc điểm nghiệp vụ của website bán đồ thể thao, hệ thống được chia thành nhiều microservice độc lập, mỗi service đảm nhận một chức năng riêng biệt nhằm đảm bảo tính mở rộng và dễ bảo trì. Các API được thiết kế theo chuẩn RESTful để phục vụ giao tiếp đồng bộ giữa client và các service, đồng thời RabbitMQ được sử dụng để xử lý các luồng giao tiếp bất đồng bộ như gửi thông báo hoặc cập nhật trạng thái đơn hàng.

Trong quá trình phát triển hệ thống, nhóm áp dụng phương pháp triển khai theo từng service độc lập nhằm dễ dàng kiểm thử và quản lý. Mỗi service được xây dựng với cơ sở dữ liệu riêng và được container hóa bằng Docker để đảm bảo khả năng triển khai

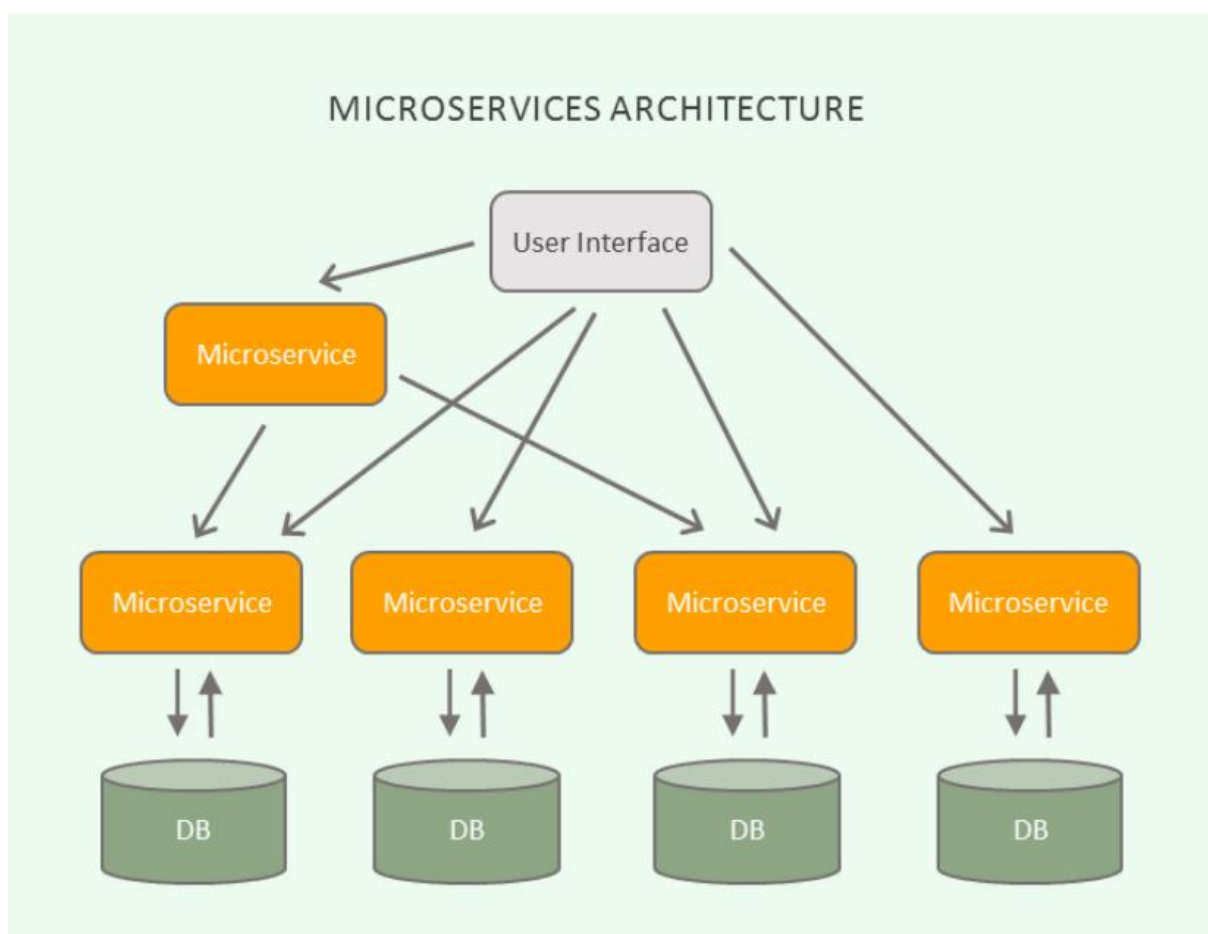
độc lập. Docker Compose được sử dụng để quản lý và khởi chạy toàn bộ hệ thống, giúp các service có thể giao tiếp với nhau trong cùng môi trường triển khai.

Sau khi hoàn thiện các chức năng chính, nhóm tiến hành kiểm thử hệ thống thông qua nhiều kịch bản nghiệp vụ khác nhau như đăng ký tài khoản, đặt hàng, thanh toán và gửi thông báo. Đồng thời, nhóm cũng thực hiện theo dõi log và đánh giá khả năng giao tiếp giữa các service nhằm kiểm tra tính ổn định của hệ thống. Kết quả kiểm thử được sử dụng để đánh giá mức độ đáp ứng của hệ thống đối với các yêu cầu của đề tài cũng như khả năng vận hành của kiến trúc Microservice trong thực tế.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về kiến trúc Microservice

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của các hệ thống phần mềm quy mô lớn, kiến trúc Microservice đã trở thành một trong những xu hướng phổ biến trong lĩnh vực phát triển phần mềm hiện đại. Các hệ thống phần mềm hiện nay thường có quy mô lớn và yêu cầu cập nhật liên tục, khiến kiến trúc Monolithic dần bộc lộ nhiều hạn chế trong triển khai và bảo trì hệ thống.



Hình 1.1. Mô hình kiến trúc Microservice.

Khác với kiến trúc Monolithic, Microservice phân tách hệ thống thành nhiều service độc lập, trong đó mỗi service đảm nhiệm một chức năng nghiệp vụ cụ thể và giao tiếp với nhau thông qua RESTful API hoặc Message Queue. Nhờ cơ chế hoạt động độc lập, các service có thể được phát triển, triển khai và mở rộng riêng biệt mà không ảnh hưởng trực tiếp đến toàn bộ hệ thống.

Kiến trúc Microservice đặc biệt phù hợp với các hệ thống thương mại điện tử, nền tảng trực tuyến hoặc các ứng dụng có quy mô lớn và yêu cầu thay đổi thường xuyên. Việc phân tách hệ thống theo từng miền nghiệp vụ giúp nâng cao khả năng bảo trì, tối ưu tài nguyên và hỗ trợ phát triển hệ thống linh hoạt hơn trong quá trình vận hành.

1.1.1. Khái niệm Microservice

Microservice là một phong cách kiến trúc phần mềm trong đó ứng dụng được xây dựng từ nhiều service nhỏ và độc lập. Mỗi service phụ trách xử lý một chức năng nghiệp vụ cụ thể và có thể hoạt động như một tiến trình riêng biệt. Các service thường giao tiếp với nhau thông qua giao thức HTTP REST hoặc hệ thống hàng đợi thông điệp.

Trong kiến trúc Microservice, mỗi service thường quản lý dữ liệu riêng nhằm giảm sự phụ thuộc giữa các thành phần trong hệ thống. Cách tổ chức này giúp các service có thể được triển khai hoặc cập nhật độc lập mà không làm ảnh hưởng đến các service khác. Ví dụ, trong hệ thống bán hàng trực tuyến, các chức năng như quản lý người dùng, quản lý sản phẩm và quản lý đơn hàng có thể được triển khai thành các service riêng biệt.

Việc phân tách hệ thống thành nhiều service giúp các nhóm phát triển có thể làm việc song song trên các thành phần khác nhau, từ đó nâng cao hiệu quả phát triển và rút ngắn thời gian triển khai phần mềm..

1.1.2. Đặc điểm của Microservice

Kiến trúc Microservice có một số đặc điểm quan trọng như sau:

- Hệ thống được phân chia thành nhiều service theo từng miền nghiệp vụ cụ thể.
- Các service có thể được triển khai và cập nhật độc lập.
- Giao tiếp giữa các service được thực hiện thông qua RESTful API hoặc Message Queue.
- Mỗi service thường quản lý cơ sở dữ liệu riêng nhằm đảm bảo tính độc lập dữ liệu.
- Hệ thống có khả năng mở rộng linh hoạt theo từng service tùy vào tải xử lý thực tế.

Nhờ cách tổ chức này, hệ thống Microservice phù hợp với các ứng dụng có quy mô lớn, yêu cầu khả năng mở rộng cao và thường xuyên thay đổi trong quá trình phát triển..

1.1.3. Ưu điểm và nhược điểm

Kiến trúc Microservice mang lại nhiều lợi ích trong quá trình phát triển và vận hành hệ thống phần mềm hiện đại. Một số ưu điểm và nhược điểm của kiến trúc này được trình bày trong Bảng 1.1 và Bảng 1.2.

Bảng 1.1. Ưu điểm của kiến trúc Microservice.

Ưu điểm	Mô tả
Triển khai độc lập	Có thể cập nhật từng service riêng biệt
Khả năng mở rộng	Chỉ mở rộng các service cần thiết
Dễ phân chia công việc	Hỗ trợ nhiều nhóm phát triển làm việc song song
Tăng tính ổn định	Lỗi tại một service ít ảnh hưởng đến toàn hệ thống

Bảng 1.2. Nhược điểm của kiến trúc Microservice.

Nhược điểm	Mô tả
Quản lý phức tạp	Hệ thống gồm nhiều service phân tán
Khó đảm bảo nhất quán dữ liệu	Mỗi service thường có cơ sở dữ liệu riêng
Yêu cầu hạ tầng cao	Cần sử dụng thêm Gateway, Monitoring hoặc Container
Khó debugging	Luồng xử lý trải dài qua nhiều service

Mặc dù tồn tại nhiều thách thức trong triển khai và quản lý, kiến trúc Microservice vẫn được sử dụng rộng rãi trong các hệ thống hiện đại nhờ khả năng mở rộng linh hoạt và hỗ trợ phát triển độc lập giữa các thành phần của hệ thống.

1.1.4. So sánh Monolithic và Microservice

Kiến trúc Monolithic là mô hình truyền thống trong đó toàn bộ chức năng của hệ thống được xây dựng và triển khai dưới dạng một ứng dụng thống nhất. Trong mô hình này, các thành phần như giao diện người dùng, xử lý nghiệp vụ và truy cập dữ liệu

thường được tích hợp trong cùng một hệ thống. Mô hình Monolithic có ưu điểm là đơn giản trong giai đoạn phát triển ban đầu và dễ triển khai đối với các hệ thống nhỏ.

Tuy nhiên, khi hệ thống phát triển với quy mô lớn, mô hình Monolithic bắt đầu bộc lộ nhiều hạn chế. Việc thay đổi hoặc cập nhật một chức năng nhỏ có thể yêu cầu triển khai lại toàn bộ hệ thống. Ngoài ra, khả năng mở rộng của Monolithic cũng hạn chế do phải mở rộng toàn bộ ứng dụng thay vì chỉ mở rộng những thành phần cần thiết.

Ngược lại, kiến trúc Microservice phân tách hệ thống thành nhiều service nhỏ độc lập. Mỗi service có thể được phát triển, triển khai và mở rộng riêng biệt. Điều này giúp hệ thống linh hoạt hơn trong quá trình vận hành và bảo trì. Microservice cũng hỗ trợ tốt hơn cho các hệ thống lớn, có lượng người dùng cao và yêu cầu thay đổi thường xuyên.

Tuy nhiên, so với Monolithic, Microservice có độ phức tạp cao hơn trong triển khai và quản lý hệ thống. Việc giám sát nhiều service, quản lý giao tiếp mạng và xử lý dữ liệu phân tán đòi hỏi hệ thống hạ tầng mạnh hơn và đội ngũ phát triển có kinh nghiệm. Vì vậy, việc lựa chọn giữa Monolithic và Microservice cần dựa trên quy mô hệ thống, yêu cầu nghiệp vụ và nguồn lực phát triển thực tế.

1.2. RESTful API

Trong kiến trúc Microservice, các service thường cần giao tiếp với nhau để trao đổi dữ liệu và phối hợp xử lý nghiệp vụ. Một trong những phương thức giao tiếp phổ biến hiện nay là RESTful API. Đây là mô hình giao tiếp dựa trên giao thức HTTP, cho phép các service truyền nhận dữ liệu một cách đơn giản, linh hoạt và dễ mở rộng.

RESTful API giúp các service có thể hoạt động độc lập nhưng vẫn đảm bảo khả năng trao đổi thông tin thông qua các endpoint được định nghĩa rõ ràng. Trong các hệ thống Microservice, RESTful API thường được sử dụng cho các giao tiếp đồng bộ giữa các service..

1.2.1. Khái niệm REST

REST (Representational State Transfer) là phong cách kiến trúc dùng để xây dựng API trên nền tảng HTTP.

Trong kiến trúc REST, mọi dữ liệu đều được xem như tài nguyên (resource) và được định danh thông qua URI riêng biệt. Client có thể truy cập và thao tác với tài nguyên bằng các HTTP request gửi đến server. Dữ liệu trao đổi giữa các thành phần

thường được biểu diễn dưới dạng JSON do có cấu trúc nhẹ, dễ đọc và được hỗ trợ rộng rãi trong nhiều ngôn ngữ lập trình.

RESTful API có ưu điểm là đơn giản, dễ tích hợp và phù hợp với các hệ thống phân tán hiện đại. Nhờ những đặc điểm đó, RESTful API hiện nay được sử dụng phổ biến trong các hệ thống Microservice và ứng dụng web.

1.2.2. Nguyên tắc thiết kế RESTful API

RESTful API được xây dựng dựa trên một số nguyên tắc thiết kế nhằm đảm bảo tính nhất quán và khả năng mở rộng của hệ thống.

Trước hết, REST áp dụng mô hình Client – Server, trong đó phía client và server được tách biệt rõ ràng. Client chịu trách nhiệm hiển thị giao diện và gửi request, trong khi server xử lý nghiệp vụ và quản lý dữ liệu.

Bên cạnh đó, REST tuân theo nguyên tắc Stateless, nghĩa là server không lưu trạng thái của client giữa các lần request. Mỗi request phải chứa đầy đủ thông tin cần thiết để server xử lý yêu cầu.

Ngoài ra, RESTful API xem mọi dữ liệu là các resource riêng biệt và sử dụng HTTP Method phù hợp để thao tác dữ liệu. Các API cũng cần được thiết kế theo giao diện thống nhất nhằm giúp client dễ dàng tích hợp và sử dụng.

Việc tuân thủ các nguyên tắc RESTful giúp hệ thống tăng khả năng mở rộng, hỗ trợ bảo trì và cải thiện hiệu quả giao tiếp giữa các service trong kiến trúc Microservice.

1.2.3. HTTP Methods

Trong RESTful API, các thao tác trên tài nguyên được thực hiện thông qua các phương thức HTTP (HTTP Methods). Mỗi phương thức thể hiện một hành động cụ thể đối với dữ liệu trên server.

Bảng 1.3. Các HTTP Method phổ biến.

Method	Chức năng
GET	Lấy dữ liệu
POST	Tạo dữ liệu mới

PUT	Cập nhật toàn bộ dữ liệu
PATCH	Cập nhật một phần dữ liệu
DELETE	Xóa dữ liệu

Việc sử dụng đúng HTTP Method giúp API trở nên rõ ràng, dễ hiểu và tuân thủ đúng nguyên tắc thiết kế RESTful.

1.2.4. HTTP Status Code

HTTP Status Code là các mã trạng thái được server trả về nhằm thông báo kết quả xử lý request của client. Việc sử dụng đúng mã trạng thái giúp client dễ dàng xác định yêu cầu thành công hay thất bại, đồng thời hỗ trợ quá trình kiểm tra và xử lý lỗi trong hệ thống.

Bảng 1.4. Các nhóm HTTP Status Code phổ biến.

Nhóm mã	Ý nghĩa	Ví dụ
2xx	Xử lý thành công	200 OK, 201 Created
4xx	Lỗi phía client	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Lỗi phía server	500 Internal Server Error, 503 Service Unavailable

Trong hệ thống Microservice, việc sử dụng chính xác HTTP Status Code có vai trò rất quan trọng vì giúp các service giao tiếp hiệu quả hơn, hỗ trợ logging, debugging và tăng khả năng xử lý lỗi trong quá trình vận hành hệ thống.

1.3. Message Queue

Trong kiến trúc Microservice, các service thường hoạt động độc lập và cần trao đổi dữ liệu với nhau trong quá trình xử lý nghiệp vụ. Nếu tất cả các service đều giao tiếp trực tiếp thông qua RESTful API thì hệ thống dễ gặp tình trạng phụ thuộc chặt chẽ giữa các service, làm giảm khả năng mở rộng và tăng nguy cơ gián đoạn khi một service gặp

lỗi. Để giải quyết vấn đề này, nhiều hệ thống hiện nay sử dụng Message Queue nhằm hỗ trợ cơ chế giao tiếp bất đồng bộ giữa các service.

Message Queue đóng vai trò trung gian trong việc truyền tải dữ liệu giữa các thành phần của hệ thống. Thay vì giao tiếp trực tiếp với nhau, các service có thể gửi và nhận thông điệp thông qua hàng đợi. Điều này giúp giảm sự phụ thuộc giữa các service, tăng khả năng chịu lỗi và cải thiện hiệu năng xử lý của hệ thống phân tán.

1.3.1. Khái niệm Message Queue

Message Queue là cơ chế lưu trữ và truyền tải thông điệp giữa các thành phần trong hệ thống thông qua một hàng đợi trung gian. Khi một service cần gửi dữ liệu hoặc sự kiện cho service khác, dữ liệu đó sẽ được đóng gói thành message và đưa vào queue. Service nhận sẽ lấy message từ queue để xử lý vào thời điểm thích hợp.

Cơ chế này cho phép các service hoạt động độc lập với nhau mà không cần kết nối trực tiếp hoặc chờ phản hồi ngay lập tức. Nhờ đó, hệ thống có thể xử lý dữ liệu theo mô hình bất đồng bộ, các tác vụ có thời gian xử lý dài có thể được xử lý nền thay vì thực hiện trực tiếp trong request chính.

Trong các hệ thống Microservice, Message Queue thường được sử dụng trong các nghiệp vụ như:

- Gửi thông báo
- Xử lý thanh toán
- Cập nhật tồn kho
- Đồng bộ dữ liệu giữa các service
- Ghi log hoặc xử lý background task

Ví dụ, khi người dùng tạo đơn hàng, Order Service có thể gửi message vào queue thay vì trực tiếp gọi Notification Service hoặc Inventory Service. Các service liên quan sẽ tự động nhận message và xử lý nghiệp vụ tương ứng.

Việc sử dụng Message Queue giúp hệ thống tăng khả năng mở rộng và hạn chế ảnh hưởng dây chuyền khi một service tạm thời gặp lỗi hoặc quá tải.

1.3.2. Mô hình *Producer – Consumer*

Message Queue thường hoạt động dựa trên mô hình *Producer – Consumer*. Trong mô hình này, *Producer* là thành phần có nhiệm vụ tạo và gửi message vào queue, trong khi *Consumer* là thành phần nhận và xử lý message từ queue.

Producer chỉ chịu trách nhiệm gửi message mà không cần quan tâm service nào sẽ xử lý message đó. Sau khi message được đưa vào queue, hệ thống hàng đợi sẽ lưu trữ message cho đến khi *Consumer* lấy ra để xử lý. Điều này giúp các service giao tiếp thông qua queue mà không cần kết nối trực tiếp với nhau.

Trong kiến trúc *Microservice*, một service có thể đóng vai trò *Producer* hoặc *Consumer* tùy thuộc vào nghiệp vụ cụ thể. Ví dụ, *Order Service* gửi thông tin đơn hàng vào queue, trong khi *Notification Service* nhận message để gửi email và *Inventory Service* nhận message để cập nhật tồn kho.

Mô hình *Producer – Consumer* hỗ trợ xử lý bất đồng bộ, giảm tải cho các service chính và giúp hệ thống dễ dàng mở rộng khi số lượng request tăng cao..

1.3.3. *RabbitMQ / Kafka / Redis Stream*

Hiện nay có nhiều công nghệ hỗ trợ triển khai Message Queue trong các hệ thống *Microservice*, trong đó phổ biến nhất là *RabbitMQ*, *Apache Kafka* và *Redis Stream*.

RabbitMQ là message broker được xây dựng dựa trên giao thức *AMQP* (*Advanced Message Queuing Protocol*). *RabbitMQ* hỗ trợ nhiều mô hình queue khác nhau, có khả năng định tuyến message linh hoạt và phù hợp với các hệ thống vừa và nhỏ.

Apache Kafka là nền tảng xử lý dữ liệu phân tán có khả năng xử lý lượng lớn message với hiệu năng cao. *Kafka* thường được sử dụng trong các hệ thống yêu cầu xử lý dữ liệu thời gian thực hoặc streaming dữ liệu liên tục.

Redis Stream là tính năng của *Redis* cho phép xử lý message theo dạng luồng dữ liệu. *Redis Stream* có ưu điểm là tốc độ xử lý nhanh, cấu hình đơn giản và phù hợp với các hệ thống cần hiệu năng cao nhưng không quá phức tạp.

Mỗi công nghệ Message Queue đều có đặc điểm riêng và được lựa chọn tùy theo yêu cầu thực tế của hệ thống. Trong các dự án *Microservice* quy mô vừa và nhỏ, *RabbitMQ* thường được sử dụng phổ biến nhờ dễ triển khai và có tài liệu hỗ trợ phong phú.

1.3.4. Giao tiếp bất đồng bộ trong microservice

Trong kiến trúc Microservice, giao tiếp bất đồng bộ là cơ chế cho phép service gửi yêu cầu mà không cần chờ phản hồi ngay lập tức từ service nhận. Thay vì gọi trực tiếp thông qua RESTful API, service sẽ gửi message vào queue và tiếp tục xử lý các công việc khác. Service nhận sẽ xử lý message khi có thể. Điều này cũng giúp giảm nguy cơ toàn bộ hệ thống bị ảnh hưởng khi một service gặp lỗi hoặc quá tải.

Ví dụ, trong hệ thống bán hàng trực tuyến, sau khi người dùng tạo đơn hàng:

1. Order Service lưu thông tin đơn hàng
2. Order Service gửi message vào queue
3. Notification Service nhận message để gửi thông báo
4. Inventory Service nhận message để cập nhật tồn kho

Trong trường hợp Notification Service tạm thời không hoạt động, message vẫn được lưu trong queue cho đến khi service hoạt động trở lại. Điều này giúp đảm bảo dữ liệu không bị mất và tăng tính chịu lỗi của hệ thống.

Tuy nhiên, cơ chế này cũng làm tăng độ phức tạp trong việc quản lý message, xử lý retry và đảm bảo tính nhất quán dữ liệu giữa các service. Vì vậy, việc thiết kế luồng message và cơ chế xử lý lỗi phù hợp là rất quan trọng trong các hệ thống Microservice hiện đại.

1.4. Docker và Docker Compose

Trong quá trình phát triển hệ thống Microservice, việc triển khai nhiều service độc lập trên cùng một môi trường thường gặp khó khăn do sự khác biệt về thư viện, cấu hình và hệ điều hành. Để giải quyết vấn đề này, công nghệ container được sử dụng nhằm đóng gói ứng dụng cùng toàn bộ môi trường thực thi thành một đơn vị độc lập.

Docker là nền tảng phổ biến hỗ trợ triển khai container trong các hệ thống hiện đại. Bên cạnh đó, Docker Compose được sử dụng để quản lý và triển khai đồng thời nhiều container trong cùng một hệ thống. Việc kết hợp Docker và Docker Compose giúp quá trình triển khai Microservice trở nên đơn giản, nhất quán và dễ quản lý hơn.

1.4.1. Khái niệm container

Container là môi trường chạy ứng dụng độc lập, bao gồm toàn bộ các thành phần cần thiết để ứng dụng hoạt động như mã nguồn, thư viện, package và các tệp cấu hình. Việc đóng gói ứng dụng dưới dạng container giúp ứng dụng có thể hoạt động nhất quán trên nhiều môi trường khác nhau.

Khác với máy ảo truyền thống, container không cần cài đặt toàn bộ hệ điều hành riêng biệt mà sử dụng chung kernel của hệ điều hành máy chủ. Nhờ đó, container có kích thước nhỏ hơn, tốc độ khởi động nhanh hơn và sử dụng tài nguyên hiệu quả hơn.

Trong kiến trúc Microservice, mỗi service thường được triển khai trong một container riêng nhằm đảm bảo tính độc lập giữa các thành phần của hệ thống. Ví dụ, User Service, Product Service và Order Service có thể hoạt động trong các container khác nhau nhưng vẫn giao tiếp với nhau thông qua mạng nội bộ.

Việc sử dụng container giúp hệ thống dễ triển khai, dễ mở rộng và hạn chế các lỗi phát sinh do khác biệt cấu hình môi trường.

1.4.2. Docker Compose

Docker Compose là công cụ hỗ trợ quản lý và triển khai nhiều container Docker thông qua một tệp cấu hình duy nhất có định dạng YAML, thường được đặt tên là `docker-compose.yml`.

Trong hệ thống Microservice, Docker Compose cho phép định nghĩa toàn bộ các service, network và cấu hình môi trường trong cùng một file. Điều này giúp việc triển khai hệ thống trở nên đơn giản và nhất quán hơn.

Mỗi service trong Docker Compose có thể được cấu hình các thông tin như image, cổng kết nối, biến môi trường, volume lưu trữ và network nội bộ. Khi cần triển khai hệ thống, lập trình viên chỉ cần sử dụng một lệnh để khởi động toàn bộ các service.

Bên cạnh đó, Docker Compose còn hỗ trợ việc dừng, khởi động lại hoặc cập nhật các service một cách thuận tiện. Nhờ khả năng quản lý tập trung, Docker Compose hiện nay được sử dụng phổ biến trong các dự án Microservice quy mô nhỏ và trung bình.

1.4.3. Triển khai nhiều service

Một trong những đặc điểm quan trọng của kiến trúc Microservice là khả năng triển khai nhiều service độc lập trong cùng một hệ thống. Mỗi service có thể được triển khai và cập nhật riêng theo nhu cầu thực tế.

Trong quá trình triển khai, mỗi microservice thường hoạt động trong một container riêng biệt và giao tiếp với nhau thông qua mạng nội bộ của Docker. Ví dụ, API Gateway có thể gửi request đến User Service hoặc Product Service thông qua tên service đã được cấu hình trong Docker Compose.

Việc triển khai nhiều service bằng Docker giúp hệ thống dễ dàng mở rộng theo nhu cầu thực tế. Khi một service có tải xử lý lớn, hệ thống chỉ cần tăng số lượng instance của service đó thay vì mở rộng toàn bộ ứng dụng.

Ngoài ra, việc triển khai độc lập các service còn giúp quá trình bảo trì và cập nhật hệ thống trở nên linh hoạt hơn. Khi cần sửa lỗi hoặc nâng cấp một service, chỉ cần triển khai lại container tương ứng mà không làm gián đoạn toàn bộ hệ thống.

1.5. JWT Authentication

Trong các hệ thống Microservice, việc xác thực người dùng và bảo mật API đóng vai trò quan trọng nhằm đảm bảo dữ liệu chỉ được truy cập bởi những người dùng hợp lệ. Do các service thường hoạt động độc lập và giao tiếp thông qua mạng, hệ thống cần có cơ chế xác thực phù hợp để kiểm tra danh tính người dùng trước khi cho phép truy cập tài nguyên.

Một trong những giải pháp xác thực phổ biến hiện nay là JWT (JSON Web Token). JWT cho phép xác thực người dùng thông qua token mà không cần lưu trữ session trên server. Điều này đặc biệt phù hợp với kiến trúc Microservice vì các service có thể xác thực người dùng độc lập mà không cần chia sẻ trạng thái phiên làm việc.

1.5.1. Khái niệm JWT

JWT (JSON Web Token) là chuẩn mở dùng để truyền tải thông tin xác thực giữa client và server dưới dạng chuỗi token đã được ký số. JWT thường được sử dụng trong các hệ thống RESTful API nhằm xác thực người dùng và hỗ trợ phân quyền truy cập tài nguyên.

Một JWT thông thường bao gồm ba thành phần chính là Header, Payload và Signature. Header chứa thông tin về loại token và thuật toán mã hóa được sử dụng. Payload lưu trữ thông tin liên quan đến người dùng như mã định danh hoặc quyền truy cập. Signature được tạo nhằm đảm bảo tính toàn vẹn của token và ngăn chặn việc sửa đổi dữ liệu.

Sau khi người dùng đăng nhập thành công, server sẽ tạo JWT và gửi về cho client. Trong các request tiếp theo, client sẽ gửi token kèm theo trong Authorization Header để server xác thực người dùng trước khi xử lý request.

JWT có ưu điểm là không cần lưu session trên server, hỗ trợ xác thực nhanh và phù hợp với các hệ thống phân tán hiện đại.

1.5.2. Quy trình xác thực

Quy trình xác thực bằng JWT thường bắt đầu khi người dùng gửi thông tin đăng nhập đến hệ thống thông qua API đăng nhập. Server sẽ kiểm tra thông tin tài khoản và mật khẩu trong cơ sở dữ liệu. Nếu thông tin hợp lệ, server sẽ tạo JWT và gửi token về cho client.

Sau khi nhận được token, client sẽ lưu token và gửi kèm token trong mỗi request tiếp theo thông qua Authorization Header. Khi server nhận request, hệ thống sẽ kiểm tra tính hợp lệ của token bằng cách xác minh chữ ký số và thời gian hết hạn của token.

Nếu token hợp lệ, server sẽ cho phép người dùng truy cập tài nguyên hoặc thực hiện các thao tác nghiệp vụ tương ứng. Ngược lại, nếu token không hợp lệ hoặc đã hết hạn, hệ thống sẽ từ chối request và trả về mã lỗi xác thực như 401 Unauthorized.

Trong kiến trúc Microservice, JWT giúp các service có thể xác thực người dùng độc lập mà không cần chia sẻ session hoặc trạng thái xác thực chung.

1.5.3. Bảo mật API

Trong các hệ thống RESTful API, bảo mật API là yêu cầu quan trọng nhằm ngăn chặn truy cập trái phép và bảo vệ dữ liệu của hệ thống. JWT là một trong những cơ chế được sử dụng phổ biến để tăng cường bảo mật cho API thông qua việc xác thực người dùng bằng token.

Khi sử dụng JWT, mọi request gửi đến API đều cần kèm theo token hợp lệ. Điều này giúp hệ thống xác định chính xác danh tính người dùng trước khi xử lý request.

Ngoài chức năng xác thực, JWT còn hỗ trợ phân quyền truy cập dựa trên thông tin được lưu trong payload của token.

Bên cạnh JWT, hệ thống thường kết hợp thêm nhiều giải pháp bảo mật khác như mã hóa mật khẩu bằng bcrypt, sử dụng giao thức HTTPS, giới hạn thời gian hết hạn của token và kiểm tra quyền truy cập trước khi xử lý request.

Trong hệ thống Microservice, JWT thường được kết hợp với HTTPS và cơ chế phân quyền để bảo vệ API và kiểm soát truy cập người dùng.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Mô tả bài toán

Trong bối cảnh các nền tảng thương mại điện tử và dịch vụ trực tuyến phát triển mạnh mẽ, yêu cầu về khả năng mở rộng, tính ổn định và tốc độ triển khai hệ thống ngày càng trở nên quan trọng. Các hệ thống được xây dựng theo mô hình Monolithic thường gặp nhiều hạn chế khi quy mô tăng lên, đặc biệt trong việc bảo trì, triển khai và mở rộng chức năng. Vì vậy, kiến trúc Microservice được lựa chọn nhằm khắc phục các hạn chế trên và nâng cao tính linh hoạt cho hệ thống.

Đề tài tập trung xây dựng hệ thống thương mại điện tử theo kiến trúc Microservice, sử dụng RESTful API kết hợp với Message Queue để triển khai cơ chế giao tiếp giữa các service. Hệ thống được phân tách thành nhiều service độc lập theo từng miền nghiệp vụ nhằm hỗ trợ triển khai và mở rộng riêng biệt.

Hệ thống bao gồm các thành phần chính như User Service, Product Service, Order Service, Payment Service, Recommendation Service và Notification Service. Trong đó, User Service quản lý thông tin người dùng và xác thực hệ thống; Product Service quản lý dữ liệu sản phẩm và nội dung hiển thị; Order Service xử lý đơn hàng; Payment Service thực hiện thanh toán trực tuyến; Recommendation Service hỗ trợ gợi ý sản phẩm; còn Notification Service gửi thông báo đến người dùng.

Việc tách các service theo từng miền nghiệp vụ giúp giảm coupling giữa các thành phần và hỗ trợ mở rộng hệ thống linh hoạt hơn. Tuy nhiên, trong phạm vi đề tài, một số chức năng như quản lý banner hoặc collection được tích hợp trực tiếp trong Product Service thay vì tách thành CMS Service riêng nhằm giảm số lượng service và hạn chế tăng độ phức tạp trong quá trình triển khai.

Hệ thống sử dụng API Gateway để tiếp nhận request từ client và định tuyến đến các service tương ứng. RabbitMQ được sử dụng làm Message Queue nhằm hỗ trợ giao tiếp bất đồng bộ giữa các service, đặc biệt đối với các nghiệp vụ như gửi email, xử lý thanh toán hoặc cập nhật tồn kho.

Về công nghệ triển khai, hệ thống sử dụng Spring Boot và Spring Cloud để xây dựng Microservice, Redis để cache dữ liệu, Elasticsearch để hỗ trợ tìm kiếm sản phẩm và Docker Compose để triển khai hệ thống dưới dạng container. Ngoài ra, Prometheus,

Grafana, Loki và Promtail được tích hợp nhằm hỗ trợ monitoring và centralized logging trong quá trình vận hành.

Mặc dù kiến trúc Microservice giúp hệ thống tăng khả năng mở rộng và triển khai độc lập, kiến trúc này cũng làm tăng độ phức tạp trong quản lý giao tiếp giữa các service và xử lý dữ liệu phân tán. Vì vậy, đề tài chỉ tập trung triển khai các thành phần cốt lõi của kiến trúc Microservice nhằm đảm bảo tính thực tiễn và phù hợp với quy mô hệ thống.

2.2. Yêu cầu hệ thống

2.2.1. Yêu cầu chức năng

Hệ thống cần cung cấp đầy đủ các chức năng phục vụ quá trình quản lý và vận hành nền tảng dịch vụ trực tuyến theo kiến trúc Microservice.

Trước hết, hệ thống cần hỗ trợ chức năng quản lý người dùng thông qua User Service. Người dùng có thể đăng ký tài khoản, đăng nhập và cập nhật thông tin cá nhân. Hệ thống sử dụng JWT để thực hiện xác thực và phân quyền truy cập đối với từng nhóm người dùng khác nhau.

Product Service cung cấp các chức năng quản lý sản phẩm như thêm mới, cập nhật, xóa và tìm kiếm sản phẩm. Người dùng có thể xem danh sách sản phẩm, tra cứu thông tin chi tiết và thực hiện các thao tác tìm kiếm dữ liệu.

Order Service chịu trách nhiệm xử lý các nghiệp vụ liên quan đến đơn hàng. Người dùng có thể tạo đơn hàng, theo dõi trạng thái xử lý và xem lịch sử giao dịch. Khi đơn hàng được tạo thành công, hệ thống sẽ phát sinh message gửi đến RabbitMQ để các service liên quan tiếp tục xử lý nghiệp vụ.

Payment Service hỗ trợ xử lý thanh toán trực tuyến và giao tiếp với các cổng thanh toán bên ngoài. Sau khi thanh toán thành công, hệ thống sẽ cập nhật trạng thái đơn hàng và gửi thông báo đến người dùng.

Notification Service có nhiệm vụ tiếp nhận message từ message broker để gửi email hoặc thông báo đến người dùng khi xảy ra các sự kiện như tạo đơn hàng thành công, thanh toán hoàn tất hoặc phát sinh lỗi hệ thống.

Ngoài ra, hệ thống còn sử dụng API Gateway để tiếp nhận và định tuyến request từ client đến các service tương ứng. Eureka Service Registry được triển khai nhằm hỗ trợ quản lý và phát hiện service trong môi trường phân tán.

2.2.2. Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo hiệu năng, tính ổn định và khả năng vận hành trong môi trường thực tế.

Trước hết, hệ thống cần đảm bảo khả năng mở rộng linh hoạt. Với kiến trúc Microservice, mỗi service có thể được triển khai và mở rộng độc lập tùy theo tải thực tế của hệ thống. Các service được container hóa bằng Docker và có thể triển khai trên Kubernetes nhằm hỗ trợ khả năng mở rộng tự động và quản lý tài nguyên hiệu quả.

Hệ thống cần đảm bảo tính sẵn sàng và khả năng chịu lỗi. Trong trường hợp một service gặp sự cố, các service còn lại vẫn phải duy trì hoạt động ổn định. RabbitMQ được sử dụng nhằm giảm sự phụ thuộc trực tiếp giữa các service và hỗ trợ xử lý bất đồng bộ trong hệ thống.

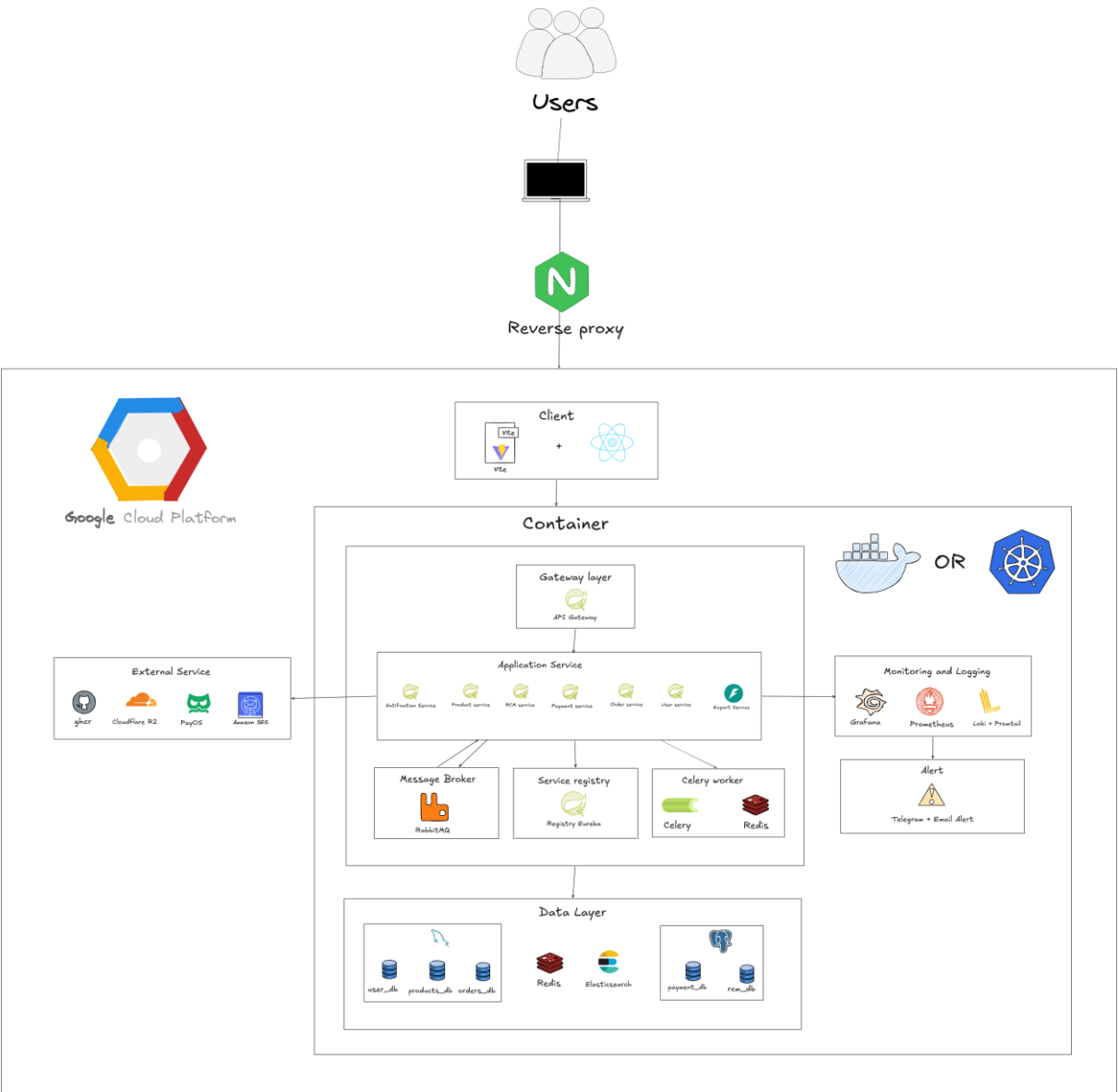
Về mặt hiệu năng, hệ thống cần hỗ trợ xử lý đồng thời nhiều request từ người dùng với thời gian phản hồi ổn định. Redis được sử dụng để cache dữ liệu nhằm tăng tốc độ truy xuất và giảm tải cho cơ sở dữ liệu chính. Elasticsearch hỗ trợ tìm kiếm dữ liệu nhanh và hiệu quả đối với các chức năng tra cứu sản phẩm hoặc nội dung.

Hệ thống cũng cần đảm bảo tính bảo mật trong quá trình giao tiếp giữa client và các service. JWT được sử dụng để xác thực người dùng và kiểm soát quyền truy cập API. API Gateway đóng vai trò kiểm soát truy cập tập trung nhằm hạn chế các request không hợp lệ.

Ngoài ra, hệ thống cần hỗ trợ monitoring và centralized logging để theo dõi hoạt động của các service. Prometheus được sử dụng để thu thập metric hệ thống, Grafana hỗ trợ trực quan hóa dữ liệu giám sát, còn Loki và Promtail được sử dụng để quản lý log tập trung. Hệ thống cũng triển khai cơ chế cảnh báo thông qua Telegram và Email Alert khi phát hiện lỗi hoặc tài nguyên vượt ngưỡng cho phép.

Cuối cùng, hệ thống cần đảm bảo khả năng triển khai linh hoạt trên nền tảng cloud nhằm nâng cao tính sẵn sàng và khả năng mở rộng trong môi trường thực tế.

2.3. Kiến trúc tổng thể hệ thống



Hình 2.1. Kiến trúc tổng quan hệ thống.

2.3.1. Sơ đồ kiến trúc microservice

Hệ thống được xây dựng theo mô hình kiến trúc Microservice nhiều tầng nhằm đảm bảo tính độc lập, khả năng mở rộng và tính dễ bảo trì. Toàn bộ hệ thống được triển khai dưới dạng container thông qua Docker hoặc Kubernetes.

Ở tầng client, người dùng truy cập hệ thống thông qua giao diện web được xây dựng bằng ReactJS kết hợp với Vite. Tất cả request từ phía client sẽ được gửi đến API Gateway trước khi chuyển tiếp đến các service tương ứng.

API Gateway đóng vai trò là cổng giao tiếp trung tâm của hệ thống. Thành phần này chịu trách nhiệm định tuyến request, kiểm tra xác thực JWT và chuyển tiếp request đến các microservice phù hợp.

Tầng Application Service bao gồm nhiều service độc lập như User Service, Product Service, Order Service, Payment Service và Notification Service. Mỗi service đảm nhiệm một miền nghiệp vụ riêng biệt và có thể hoạt động độc lập với các service khác.

RabbitMQ được sử dụng làm message broker nhằm triển khai cơ chế giao tiếp bất đồng bộ giữa các service. Khi một sự kiện phát sinh, service tương ứng sẽ publish message lên RabbitMQ để các service khác subscribe và xử lý.

Hệ thống sử dụng Eureka Service Registry nhằm hỗ trợ đăng ký và phát hiện service trong môi trường phân tán. Điều này giúp các service có thể dễ dàng tìm kiếm và giao tiếp với nhau mà không cần cấu hình địa chỉ cố định.

Tầng dữ liệu bao gồm nhiều cơ sở dữ liệu riêng biệt nhằm đảm bảo tính độc lập dữ liệu giữa các service. Hệ thống sử dụng MySQL, PostgreSQL, Redis và Elasticsearch tùy theo yêu cầu xử lý dữ liệu của từng service.

Ngoài ra, hệ thống còn tích hợp các công cụ monitoring và logging như Prometheus, Grafana, Loki và Promtail nhằm hỗ trợ giám sát và theo dõi trạng thái hoạt động của toàn hệ thống.

Mặc dù kiến trúc Microservice giúp hệ thống tăng khả năng mở rộng và hỗ trợ triển khai độc lập giữa các service, kiến trúc này cũng làm tăng độ phức tạp trong quản lý hệ thống phân tán, giao tiếp giữa các service và đảm bảo tính nhất quán dữ liệu. Trong phạm vi đề tài, hệ thống ưu tiên triển khai các thành phần cốt lõi của kiến trúc Microservice và chưa áp dụng đầy đủ các cơ chế orchestration phức tạp như Kubernetes production cluster hoặc distributed tracing nhằm phù hợp với quy mô triển khai thực tế.

2.3.2. *Luồng xử lý nghiệp vụ*

Khi người dùng truy cập hệ thống thông qua giao diện web, request đầu tiên sẽ được gửi đến API Gateway. Gateway sẽ thực hiện kiểm tra xác thực JWT trước khi chuyển tiếp request đến service phù hợp.

Trong luồng xử lý tạo đơn hàng, người dùng lựa chọn sản phẩm và gửi yêu cầu tạo đơn hàng từ phía client. Request này được chuyển đến Order Service thông qua API Gateway. Order Service tiến hành kiểm tra thông tin đơn hàng và lưu dữ liệu vào cơ sở dữ liệu tương ứng.

Sau khi đơn hàng được tạo thành công, Order Service sẽ publish một message lên RabbitMQ chứa thông tin đơn hàng. Notification Service sẽ subscribe message này để gửi email xác nhận đến người dùng. Đồng thời, Payment Service cũng có thể nhận message để thực hiện xử lý thanh toán.

Khi quá trình thanh toán hoàn tất, Payment Service sẽ cập nhật trạng thái thanh toán và gửi message thông báo đến các service liên quan. Notification Service tiếp tục gửi thông báo xác nhận giao dịch đến người dùng.

Trong suốt quá trình vận hành, các công cụ monitoring như Prometheus và Grafana sẽ liên tục theo dõi trạng thái hoạt động của các service. Nếu phát hiện lỗi hoặc tài nguyên vượt ngưỡng cho phép, hệ thống sẽ gửi cảnh báo thông qua Telegram hoặc Email Alert để quản trị viên kịp thời xử lý.

Việc kết hợp giữa RESTful API và Message Queue giúp hệ thống vừa đảm bảo khả năng xử lý đồng bộ đối với các request trực tiếp, vừa hỗ trợ xử lý bất đồng bộ hiệu quả cho các tác vụ nền và các luồng nghiệp vụ phức tạp.

2.4. Thiết kế các Microservice

2.4.1. Thiết kế API Gateway

API Gateway đóng vai trò là cổng giao tiếp trung tâm của toàn bộ hệ thống Microservice. Tất cả request từ phía client đều phải đi qua API Gateway trước khi được chuyển tiếp đến các service tương ứng. Trong hệ thống, API Gateway được xây dựng dựa trên Spring Cloud Gateway nhằm hỗ trợ định tuyến request, xác thực bảo mật và quản lý truy cập tập trung.

API Gateway chịu trách nhiệm tiếp nhận request từ client, kiểm tra JWT Token và thực hiện định tuyến đến các service thông qua cơ chế Service Discovery của Eureka. Việc triển khai xác thực tại Gateway giúp giảm tải xử lý bảo mật cho các service phía sau, đồng thời tăng tính tập trung trong quản lý truy cập hệ thống.

Ngoài ra, Gateway còn tích hợp Redis Reactive để triển khai cơ chế Rate Limiting nhằm giới hạn số lượng request từ người dùng trong một khoảng thời gian nhất định. Điều này giúp hạn chế các cuộc tấn công flood request và đảm bảo tính ổn định của hệ thống.

Thông qua Eureka Service Registry, API Gateway có thể tự động phát hiện địa chỉ của các service đang hoạt động mà không cần cấu hình cố định. Điều này giúp tăng khả năng mở rộng và linh hoạt trong môi trường phân tán.

Kiến trúc xử lý request của Gateway được mô tả theo quy trình sau:

1. Client gửi request đến API Gateway.
2. Gateway thực hiện kiểm tra JWT Token.
3. Gateway kiểm tra quyền truy cập và giới hạn request.
4. Request được định tuyến đến service phù hợp thông qua Eureka.
5. Service xử lý request và trả response về Gateway.
6. Gateway trả kết quả cho client.

Việc sử dụng API Gateway giúp hệ thống đảm bảo tính bảo mật, khả năng mở rộng và quản lý tập trung trong kiến trúc Microservice.

2.4.2. Thiết kế Service Registry

Hệ thống sử dụng Netflix Eureka làm Service Registry nhằm hỗ trợ quản lý và phát hiện các service trong môi trường phân tán.

Trong kiến trúc Microservice, địa chỉ IP hoặc port của service có thể thay đổi trong quá trình triển khai hoặc mở rộng hệ thống. Vì vậy, Eureka được sử dụng để cho phép các service tự động đăng ký và cập nhật trạng thái hoạt động của mình.

Khi một service khởi động, service đó sẽ gửi thông tin đăng ký đến Eureka Server. API Gateway và các service khác có thể truy vấn Eureka để xác định vị trí của service cần giao tiếp. Điều này giúp loại bỏ việc cấu hình địa chỉ cố định giữa các service, đồng thời nâng cao khả năng mở rộng và tính linh hoạt của hệ thống.

Bên cạnh đó, Eureka còn hỗ trợ kiểm tra trạng thái hoạt động của service thông qua cơ chế heartbeat. Nếu một service ngừng hoạt động hoặc mất kết nối, Eureka sẽ tự

động loại bỏ service đó khỏi danh sách đăng ký nhằm tránh việc định tuyến request đến service lỗi.

Việc sử dụng Service Registry giúp hệ thống dễ dàng mở rộng số lượng instance của từng service trong môi trường cloud hoặc Kubernetes mà không ảnh hưởng đến hoạt động chung của toàn hệ thống.

2.4.3. Thiết kế User Service

User Service là thành phần chịu trách nhiệm quản lý thông tin người dùng và xử lý các chức năng xác thực trong hệ thống.

Service này cung cấp các chức năng như:

- Đăng ký tài khoản
- Đăng nhập
- Cập nhật thông tin người dùng
- Quản lý quyền truy cập
- Sinh và xác thực JWT Token

User Service sử dụng MySQL làm cơ sở dữ liệu chính để lưu trữ thông tin người dùng và Redis để cache dữ liệu nhằm tăng tốc độ truy xuất. Trong quá trình đăng nhập, hệ thống sẽ kiểm tra thông tin tài khoản và mật khẩu của người dùng. Sau khi xác thực thành công, service sẽ sinh JWT Token và trả về cho client. Token này được sử dụng trong các request tiếp theo để xác thực quyền truy cập thông qua API Gateway.

Ngoài ra, User Service còn giao tiếp với RabbitMQ để gửi các sự kiện liên quan đến người dùng như đăng ký tài khoản mới hoặc thay đổi trạng thái tài khoản nhằm phục vụ các nghiệp vụ xử lý bất đồng bộ của các service khác.

User Service được tách riêng do đây là miền nghiệp vụ có mức độ bảo mật cao và thường xuyên được nhiều service khác sử dụng. Việc triển khai độc lập giúp hệ thống dễ dàng mở rộng chức năng xác thực, quản lý người dùng và giảm ảnh hưởng đến các service khác khi thay đổi logic bảo mật hoặc cơ chế phân quyền.

2.4.4. Thiết kế Product Service

Product Service chịu trách nhiệm quản lý dữ liệu và các nghiệp vụ liên quan đến sản phẩm trong hệ thống.

Các chức năng chính của service bao gồm:

- Thêm mới sản phẩm
- Cập nhật thông tin sản phẩm
- Xóa sản phẩm
- Tìm kiếm sản phẩm
- Quản lý danh mục sản phẩm

Product Service sử dụng MySQL để lưu trữ dữ liệu sản phẩm và Redis để cache dữ liệu nhằm tăng tốc độ truy xuất. Ngoài ra, hệ thống tích hợp Elasticsearch để hỗ trợ tìm kiếm dữ liệu nhanh và hiệu quả đối với các chức năng tra cứu sản phẩm.

Khi có thay đổi liên quan đến sản phẩm, Product Service sẽ publish message lên RabbitMQ để các service khác như Recommendation Service hoặc Notification Service có thể cập nhật dữ liệu tương ứng. Việc sử dụng Elasticsearch giúp cải thiện đáng kể tốc độ tìm kiếm và hỗ trợ các chức năng tìm kiếm nâng cao như tìm kiếm gần đúng, tìm kiếm theo từ khóa hoặc lọc dữ liệu theo nhiều điều kiện khác nhau.

Product Service được tách riêng do đây là service có tần suất truy xuất dữ liệu lớn và yêu cầu xử lý tìm kiếm phức tạp. Việc triển khai độc lập giúp hệ thống dễ dàng mở rộng khả năng đọc dữ liệu, tối ưu hiệu năng tìm kiếm và hạn chế ảnh hưởng đến các service nghiệp vụ khác khi dữ liệu sản phẩm tăng lên.

2.4.5. Thiết kế Order Service

Order Service chịu trách nhiệm xử lý các nghiệp vụ liên quan đến đơn hàng trong hệ thống. Service này cung cấp các chức năng như:

- Tạo đơn hàng
- Cập nhật trạng thái đơn hàng
- Theo dõi lịch sử giao dịch
- Quản lý thông tin chi tiết đơn hàng

Order Service sử dụng MySQL làm cơ sở dữ liệu chính để lưu trữ dữ liệu đơn hàng. Sau khi đơn hàng được tạo thành công, service sẽ publish message lên RabbitMQ nhằm kích hoạt các nghiệp vụ tiếp theo như thanh toán hoặc gửi thông báo.

Hệ thống được triển khai theo hướng Event-Driven Architecture nhằm giảm sự phụ thuộc trực tiếp giữa các service và hỗ trợ xử lý bất đồng bộ. Trong trường hợp Payment Service hoặc Notification Service tạm thời gặp sự cố, Order Service vẫn có thể tiếp tục xử lý request mà không làm gián đoạn toàn bộ hệ thống. Các message sẽ được lưu trữ trong RabbitMQ và xử lý lại khi service hoạt động bình thường.

Order Service được tách riêng nhằm cô lập luồng xử lý nghiệp vụ mua hàng khỏi các chức năng khác của hệ thống. Việc triển khai độc lập giúp hệ thống dễ dàng mở rộng khả năng xử lý đơn hàng, hỗ trợ triển khai cơ chế event-driven và hạn chế ảnh hưởng đến các service khác khi lưu lượng giao dịch tăng cao.

2.4.6. Thiết kế Payment Service

Payment Service chịu trách nhiệm xử lý các nghiệp vụ thanh toán trực tuyến trong hệ thống. Service này sử dụng PostgreSQL làm cơ sở dữ liệu nhằm đảm bảo tính ổn định và hỗ trợ tốt cho các giao dịch thanh toán. Các chức năng chính bao gồm:

- Xử lý thanh toán
- Cập nhật trạng thái giao dịch
- Kiểm tra lịch sử thanh toán
- Gửi kết quả thanh toán đến các service liên quan

Payment Service giao tiếp với RabbitMQ để nhận message từ Order Service và xử lý thanh toán theo cơ chế bất đồng bộ. Sau khi giao dịch hoàn tất, service sẽ gửi message cập nhật trạng thái đến Notification Service hoặc các service liên quan khác.

Payment Service được tách riêng nhằm tăng tính bảo mật và hỗ trợ mở rộng độc lập cho hệ thống thanh toán. Ngoài ra, việc triển khai riêng service này cũng giúp hệ thống dễ dàng tích hợp thêm nhiều cổng thanh toán khác nhau trong tương lai mà không ảnh hưởng đến các service nghiệp vụ khác.

2.4.7. Thiết kế Recommendation Service

Recommendation Service được xây dựng nhằm hỗ trợ chức năng gợi ý sản phẩm và cá nhân hóa trải nghiệm người dùng.

Service này sử dụng PostgreSQL để lưu trữ dữ liệu phân tích và Redis để cache dữ liệu truy cập thường xuyên. Recommendation Service tiếp nhận dữ liệu từ Product Service hoặc Order Service thông qua RabbitMQ để phân tích hành vi người dùng và xây dựng danh sách gợi ý phù hợp.

Việc tách riêng Recommendation Service giúp hệ thống dễ dàng mở rộng các thuật toán phân tích dữ liệu hoặc tích hợp các mô hình Machine Learning trong tương lai mà không ảnh hưởng đến hoạt động của các service nghiệp vụ chính.

2.4.8. Thiết kế Notification Service

Notification Service chịu trách nhiệm gửi thông báo đến người dùng khi xảy ra các sự kiện trong hệ thống.

Service này hoạt động theo mô hình event-driven và sử dụng RabbitMQ để tiếp nhận message từ các service khác. Các thông báo có thể bao gồm:

- Xác nhận đăng ký tài khoản
- Xác nhận tạo đơn hàng
- Kết quả thanh toán
- Thông báo lỗi hệ thống

Notification Service được thiết kế độc lập nhằm tránh ảnh hưởng đến hiệu năng xử lý của các service nghiệp vụ chính. Việc xử lý thông báo bất đồng bộ giúp hệ thống giảm thời gian phản hồi đối với request từ người dùng. Ngoài ra, service còn có thể mở rộng để hỗ trợ nhiều hình thức thông báo khác nhau như Email, SMS hoặc Telegram trong tương lai.

2.4.9. Thiết kế giao tiếp giữa các Microservice

Hệ thống sử dụng kết hợp hai cơ chế giao tiếp gồm RESTful API và Message Queue nhằm đảm bảo tính linh hoạt và hiệu quả trong xử lý nghiệp vụ.

RESTful API được sử dụng cho các request cần phản hồi trực tiếp và xử lý đồng bộ giữa client với service hoặc giữa các service với nhau. Trong khi đó, RabbitMQ được sử dụng cho các nghiệp vụ bất đồng bộ nhằm giảm sự phụ thuộc trực tiếp giữa các service.

Mô hình giao tiếp bất đồng bộ giúp hệ thống:

- Tăng khả năng mở rộng
- Giảm coupling giữa các service
- Nâng cao khả năng chịu lỗi
- Hỗ trợ xử lý tác vụ nền hiệu quả

Bên cạnh đó, việc sử dụng RabbitMQ còn giúp hệ thống đảm bảo tính ổn định khi lưu lượng request tăng cao hoặc khi một service tạm thời gặp sự cố.

2.4.10. Thiết kế cơ sở dữ liệu phân tán

Hệ thống được thiết kế theo mô hình “Database per Service”, trong đó mỗi microservice sở hữu cơ sở dữ liệu riêng nhằm đảm bảo tính độc lập dữ liệu.

Các service như User Service, Product Service và Order Service sử dụng MySQL để lưu trữ dữ liệu quan hệ. Trong khi đó, Payment Service và Recommendation Service sử dụng PostgreSQL nhằm hỗ trợ tốt hơn cho các nghiệp vụ đặc thù. Redis được sử dụng làm hệ thống cache nhằm tăng tốc độ truy xuất dữ liệu và giảm tải cho cơ sở dữ liệu chính. Elasticsearch được triển khai để hỗ trợ tìm kiếm dữ liệu tốc độ cao đối với Product Service.

Việc tách riêng cơ sở dữ liệu cho từng service giúp hệ thống dễ dàng mở rộng, triển khai độc lập và hạn chế ảnh hưởng lẫn nhau giữa các miền nghiệp vụ.

2.4.11. Thiết kế triển khai hệ thống bằng Docker

Toàn bộ hệ thống được đóng gói và triển khai dưới dạng container thông qua Docker nhằm đảm bảo tính nhất quán giữa các môi trường phát triển và triển khai thực tế.

Mỗi microservice được xây dựng thành một container độc lập với Dockerfile riêng. Docker Compose được sử dụng để quản lý và khởi chạy toàn bộ hệ thống bao gồm:

- API Gateway
- Eureka Service Registry
- Các microservice nghiệp vụ
- MySQL
- PostgreSQL
- Redis
- RabbitMQ
- Elasticsearch

Các container được kết nối thông qua Docker Network nhằm hỗ trợ giao tiếp nội bộ giữa các service. Ngoài ra, Docker Volume được sử dụng để lưu trữ dữ liệu persistent cho các hệ quản trị cơ sở dữ liệu và message broker.

Việc triển khai bằng Docker giúp hệ thống dễ dàng mở rộng, triển khai trên cloud và hỗ trợ quá trình tích hợp liên tục (CI/CD) trong môi trường phát triển hiện đại.

2.5. Thiết kế cơ sở dữ liệu

Trong kiến trúc Microservice, cơ sở dữ liệu được thiết kế theo hướng phân tách dữ liệu theo từng miền nghiệp vụ nhằm đảm bảo tính độc lập giữa các service. Thay vì sử dụng một cơ sở dữ liệu tập trung cho toàn bộ hệ thống, mỗi microservice sở hữu cơ sở dữ liệu riêng và chịu trách nhiệm quản lý dữ liệu thuộc phạm vi nghiệp vụ của mình. Mô hình này giúp giảm sự phụ thuộc giữa các service, hỗ trợ triển khai độc lập và nâng cao khả năng mở rộng của hệ thống.

Hệ thống sử dụng kết hợp hai hệ quản trị cơ sở dữ liệu quan hệ là MySQL và PostgreSQL. MySQL được sử dụng cho các service có tần suất truy xuất dữ liệu lớn như User Service, Product Service và Order Service. Trong khi đó, PostgreSQL được sử dụng cho Payment Service và Recommendation Service nhằm hỗ trợ tốt hơn cho các nghiệp vụ liên quan đến giao dịch và phân tích dữ liệu.

Ngoài ra, Redis được sử dụng làm hệ thống cache nhằm tăng tốc độ truy xuất dữ liệu, còn Elasticsearch được tích hợp để hỗ trợ tìm kiếm dữ liệu sản phẩm với hiệu năng cao.

2.5.1. Thiết kế cơ sở dữ liệu User Service

User Service sử dụng cơ sở dữ liệu `users_db` để lưu trữ thông tin người dùng và dữ liệu xác thực hệ thống. Cơ sở dữ liệu này bao gồm các bảng chính như: `users`, `roles`, `user_roles`, `refresh_tokens`.

Bảng `users` lưu trữ các thông tin cơ bản của người dùng như tên đăng nhập, email, mật khẩu đã mã hóa và trạng thái tài khoản. Bảng `roles` quản lý các vai trò trong hệ thống như quản trị viên hoặc người dùng thông thường. Do một người dùng có thể sở hữu nhiều vai trò khác nhau, bảng `user_roles` được sử dụng để biểu diễn quan hệ nhiều-nhiều giữa người dùng và vai trò. Ngoài ra, bảng `refresh_tokens` hỗ trợ cơ chế xác thực JWT bằng cách lưu trữ refresh token phục vụ quá trình gia hạn phiên đăng nhập.

Thiết kế này giúp User Service quản lý độc lập toàn bộ dữ liệu liên quan đến xác thực và phân quyền người dùng, đồng thời hỗ trợ mở rộng các cơ chế bảo mật trong tương lai.

2.5.2. Thiết kế cơ sở dữ liệu Product Service

Product Service sử dụng cơ sở dữ liệu `products_db` để quản lý toàn bộ dữ liệu liên quan đến sản phẩm và nội dung hiển thị. Các bảng chính bao gồm `categories`, `sub_categories`, `brands`, `colors`, `sizes`, `topics`, `collections`, `products`, `product_variants`, `product_images`, `product_collections`, `product_topics`, `inventory`, `banners`, `store_settings` và `product_similarity`.

Bảng `products` lưu trữ thông tin cơ bản của sản phẩm như tên, mô tả, giá bán và trạng thái hiển thị. Các bảng `categories` và `sub_categories` dùng để phân loại sản phẩm theo danh mục. Bảng `brands` quản lý thông tin thương hiệu, trong khi các bảng `colors` và `sizes` hỗ trợ quản lý thuộc tính sản phẩm.

Đối với các sản phẩm có nhiều phiên bản, bảng `product_variants` được sử dụng để lưu trữ thông tin biến thể theo màu sắc, kích thước hoặc các thuộc tính khác. Bảng `product_images` lưu trữ thông tin hình ảnh của sản phẩm. Bảng `inventory` quản lý số lượng tồn kho tương ứng với từng sản phẩm hoặc biến thể sản phẩm.

Ngoài ra, Product Service còn quản lý các bảng phục vụ nội dung hiển thị như `banners`, `store_settings`, `topics` và `collections`. Bảng `product_similarity` được sử dụng để

lưu trữ dữ liệu độ tương đồng giữa các sản phẩm, phục vụ cho chức năng gợi ý hoặc đề xuất sản phẩm liên quan.

2.5.3. Thiết kế cơ sở dữ liệu Order Service

Order Service sử dụng cơ sở dữ liệu `orders_db` để lưu trữ các dữ liệu liên quan đến quá trình mua hàng và xử lý đơn hàng. Các bảng chính bao gồm `discounts`, `carts`, `cart_items`, `wishlists`, `wishlist_items`, `orders`, `order_items`, `discount_usages`, `payment_methods`, `payments` và `reviews`.

Bảng `carts` và `cart_items` được sử dụng để quản lý giỏ hàng của người dùng. Trong đó, `carts` lưu thông tin giỏ hàng tổng quát, còn `cart_items` lưu danh sách sản phẩm trong giỏ hàng. Tương tự, `wishlists` và `wishlist_items` dùng để lưu trữ danh sách sản phẩm yêu thích của người dùng.

Bảng `orders` lưu trữ thông tin đơn hàng như mã đơn hàng, người dùng đặt hàng, tổng tiền và trạng thái đơn hàng. Bảng `order_items` lưu trữ chi tiết từng sản phẩm thuộc một đơn hàng. Các bảng `discounts` và `discount_usages` hỗ trợ chức năng mã giảm giá, bao gồm thông tin khuyến mãi và lịch sử sử dụng mã giảm giá của người dùng.

Ngoài ra, `orders_db` còn có các bảng `payment_methods` và `payments` để lưu trữ dữ liệu thanh toán trong giai đoạn ban đầu hoặc phục vụ quá trình chuyển đổi dữ liệu sang Payment Service. Bảng `reviews` được sử dụng để lưu trữ đánh giá của người dùng đối với sản phẩm sau khi mua hàng.

2.5.4. Thiết kế cơ sở dữ liệu Payment Service

Payment Service sử dụng cơ sở dữ liệu `payments_db` trên PostgreSQL để quản lý dữ liệu thanh toán. Các bảng chính bao gồm `payment_methods` và `payments`.

Bảng `payment_methods` lưu trữ danh sách các phương thức thanh toán được hệ thống hỗ trợ, chẳng hạn như thanh toán khi nhận hàng hoặc thanh toán trực tuyến. Bảng `payments` lưu trữ thông tin giao dịch thanh toán, bao gồm mã đơn hàng, số tiền thanh toán, phương thức thanh toán, mã giao dịch, trạng thái thanh toán và thời gian thanh toán.

Trong thiết kế Microservice, Payment Service không phụ thuộc trực tiếp vào cơ sở dữ liệu của Order Service. Tuy nhiên, để phục vụ nhu cầu tra cứu và đảm bảo tính độc lập dữ liệu, bảng `payments` có thể lưu thêm một số thông tin tham chiếu như mã đơn

hàng, email người dùng hoặc mã người dùng. Điều này giúp Payment Service có đủ dữ liệu cần thiết để xử lý nghiệp vụ mà không cần truy vấn trực tiếp sang cơ sở dữ liệu của service khác.

2.5.5. Thiết kế cơ sở dữ liệu Recommendation Service

Recommendation Service sử dụng cơ sở dữ liệu recommendations_db trên PostgreSQL để lưu trữ dữ liệu phục vụ chức năng gợi ý sản phẩm. Bảng quan trọng trong cơ sở dữ liệu này là product_similarity.

Bảng product_similarity lưu trữ mức độ tương đồng giữa hai sản phẩm thông qua các trường như mã sản phẩm thứ nhất, mã sản phẩm thứ hai, điểm tương đồng và thời gian tạo/cập nhật dữ liệu. Dữ liệu này có thể được sử dụng để xây dựng chức năng đề xuất sản phẩm liên quan, giúp cải thiện trải nghiệm người dùng và hỗ trợ hoạt động bán hàng.

Việc tách riêng dữ liệu gợi ý sản phẩm sang Recommendation Service giúp hệ thống dễ dàng mở rộng các thuật toán đề xuất trong tương lai, đồng thời hạn chế ảnh hưởng đến Product Service khi xử lý các tác vụ phân tích dữ liệu.

2.5.6. Thiết kế phân tán dữ liệu giữa các service

Hệ thống được thiết kế theo nguyên tắc “Database per Service”, trong đó mỗi microservice sở hữu cơ sở dữ liệu riêng và không truy cập trực tiếp vào cơ sở dữ liệu của service khác.

Trong trường hợp cần trao đổi dữ liệu giữa các service, hệ thống sử dụng RESTful API hoặc Message Queue để truyền dữ liệu thông qua các cơ chế giao tiếp đồng bộ và bất đồng bộ. Cách tiếp cận này giúp giảm coupling giữa các service, nâng cao khả năng mở rộng và tăng tính chịu lỗi của hệ thống.

Ví dụ, Order Service không truy cập trực tiếp vào cơ sở dữ liệu của Payment Service để kiểm tra trạng thái thanh toán. Thay vào đó, các service sẽ giao tiếp với nhau thông qua API hoặc RabbitMQ nhằm đảm bảo tính độc lập dữ liệu và tuân thủ đúng nguyên tắc thiết kế của kiến trúc Microservice.

2.6. Thiết kế RESTful API

2.6.1. Tổng quan thiết kế RESTful API

Hệ thống được xây dựng theo kiến trúc Microservice và sử dụng RESTful API làm cơ chế giao tiếp chính giữa client với các service cũng như giữa các service với nhau. Việc áp dụng RESTful API giúp chuẩn hóa phương thức trao đổi dữ liệu, đảm bảo tính nhất quán trong thiết kế và hỗ trợ khả năng mở rộng hệ thống trong tương lai.

Toàn bộ request từ phía client đều được gửi thông qua API Gateway trước khi được định tuyến đến các microservice tương ứng. API Gateway đóng vai trò kiểm tra xác thực JWT, phân quyền truy cập và chuyển tiếp request đến đúng service xử lý. Các API được thiết kế theo chuẩn RESTful với cấu trúc URL phản ánh rõ tài nguyên cần thao tác, đồng thời sử dụng versioning thông qua tiền tố /api/v1/ nhằm hỗ trợ nâng cấp hệ thống trong tương lai mà không ảnh hưởng đến các client hiện tại.

Hệ thống sử dụng các phương thức HTTP tương ứng với từng thao tác xử lý dữ liệu:

- GET dùng để truy xuất dữ liệu
- POST dùng để tạo mới dữ liệu
- PUT dùng để cập nhật dữ liệu
- PATCH dùng để cập nhật một phần dữ liệu
- DELETE dùng để xóa dữ liệu

Ngoài các API phục vụ người dùng bên ngoài, hệ thống còn sử dụng nhóm endpoint /internal/* để phục vụ giao tiếp nội bộ giữa các microservice thông qua Feign Client. Các endpoint này không được expose ra Internet nhằm đảm bảo tính bảo mật trong hệ thống phân tán.

2.6.2. Phân loại API theo cơ chế xác thực

Để đảm bảo tính bảo mật và kiểm soát quyền truy cập, hệ thống phân chia API thành nhiều nhóm khác nhau dựa trên cơ chế xác thực và phân quyền.

Nhóm Public API bao gồm các endpoint không yêu cầu JWT Token và cho phép người dùng truy cập công khai. Các API này chủ yếu phục vụ các chức năng cơ bản như đăng ký tài khoản, đăng nhập, xác thực OTP, quên mật khẩu, xem sản phẩm, tìm kiếm

sản phẩm hoặc tra cứu đơn hàng. Việc cho phép truy cập công khai đối với các API này giúp người dùng có thể sử dụng các chức năng cơ bản của hệ thống mà không cần xác thực trước.

Đối với các nghiệp vụ liên quan đến dữ liệu cá nhân hoặc thao tác mua hàng, hệ thống sử dụng nhóm API yêu cầu JWT Token. Các request thuộc nhóm này cần gửi kèm Authorization: Bearer <token> để API Gateway thực hiện xác thực trước khi chuyển tiếp đến service xử lý. Nhóm API này bao gồm các chức năng như quản lý giỏ hàng, đặt hàng, cập nhật thông tin cá nhân, thay đổi mật khẩu, quản lý danh sách yêu thích và gửi đánh giá sản phẩm.

Bên cạnh đó, hệ thống còn triển khai nhóm Admin API dành riêng cho quản trị viên. Các endpoint quản trị được đặt dưới nhóm /api/v1/admin/* và yêu cầu người dùng phải có vai trò ADMIN trong JWT Token. Nhóm API này hỗ trợ các chức năng quản trị sản phẩm, quản lý đơn hàng, quản lý người dùng, quản lý mã giảm giá và thống kê hệ thống.

Ngoài ra, hệ thống còn sử dụng Internal API cho giao tiếp nội bộ giữa các service. Các endpoint này chỉ phục vụ trao đổi dữ liệu giữa các microservice thông qua Feign Client và không cho phép client bên ngoài truy cập trực tiếp.

2.6.3. Thiết kế API của User Service

User Service cung cấp các API phục vụ xác thực và quản lý thông tin người dùng. Các API xác thực bao gồm đăng ký tài khoản, đăng nhập, xác thực OTP, làm mới access token, quên mật khẩu và đăng xuất. Hệ thống sử dụng JWT để quản lý phiên đăng nhập và phân quyền truy cập giữa các người dùng.

Ngoài các chức năng xác thực, User Service còn hỗ trợ các API quản lý hồ sơ cá nhân như xem thông tin người dùng, cập nhật profile và thay đổi mật khẩu. Đối với quản trị viên, service cung cấp các API quản lý tài khoản người dùng như lọc danh sách user, khóa tài khoản, mở khóa tài khoản và thống kê số lượng người dùng hoạt động.

Bên cạnh đó, User Service còn cung cấp Internal API nhằm hỗ trợ các service khác truy xuất thông tin người dùng thông qua user ID hoặc email mà không cần truy cập trực tiếp vào cơ sở dữ liệu của service này.

2.6.4. Thiết kế API của Product Service

Product Service là service có số lượng endpoint lớn nhất trong hệ thống do chịu trách nhiệm quản lý toàn bộ dữ liệu sản phẩm và nội dung hiển thị.

Các API của Product Service được chia thành nhiều nhóm chức năng như quản lý sản phẩm, tìm kiếm sản phẩm, quản lý danh mục, quản lý thương hiệu, quản lý biến thể sản phẩm, quản lý tồn kho và quản lý banner quảng bá. Đối với người dùng thông thường, service cung cấp các API duyệt sản phẩm, xem chi tiết sản phẩm, lọc dữ liệu và tìm kiếm full-text thông qua Elasticsearch.

Đối với quản trị viên, Product Service hỗ trợ đầy đủ các chức năng CRUD nhằm quản lý sản phẩm, hình ảnh, biến thể, danh mục và dữ liệu hiển thị trên hệ thống. Ngoài ra, Product Service còn cung cấp Internal API phục vụ giao tiếp với các service khác như Order Service hoặc Recommendation Service trong quá trình xử lý tồn kho và truy xuất dữ liệu sản phẩm.

Thiết kế API của Product Service giúp hệ thống dễ dàng mở rộng số lượng sản phẩm, hỗ trợ hiệu quả cho chức năng tìm kiếm và đảm bảo khả năng quản lý dữ liệu linh hoạt trong môi trường thương mại điện tử.

2.6.5. Thiết kế API của Order Service

Order Service cung cấp các API liên quan đến giỏ hàng, đặt hàng và quản lý giao dịch mua bán.

Hệ thống hỗ trợ nhiều luồng đặt hàng khác nhau như đặt hàng từ toàn bộ giỏ hàng, đặt hàng từ các sản phẩm được chọn hoặc mua ngay một sản phẩm cụ thể. Ngoài ra, service còn hỗ trợ các API theo dõi trạng thái đơn hàng, hủy đơn hàng và đặt lại đơn hàng đã mua trước đó.

Bên cạnh các nghiệp vụ đặt hàng, Order Service còn cung cấp các API quản lý danh sách yêu thích, đánh giá sản phẩm, mã giảm giá và tính phí vận chuyển. Đối với quản trị viên, service hỗ trợ các API quản lý trạng thái đơn hàng, xuất dữ liệu đơn hàng và thống kê doanh thu theo nhiều tiêu chí khác nhau.

Ngoài ra, Order Service còn triển khai các Internal API nhằm cung cấp dữ liệu tương tác người dùng cho Recommendation Service phục vụ quá trình xây dựng hệ thống gợi ý sản phẩm.

2.6.6. Thiết kế API của Payment Service

Payment Service cung cấp các API phục vụ xử lý thanh toán trực tuyến và quản lý phương thức thanh toán.

Các API công khai của service bao gồm webhook nhận callback từ cổng thanh toán PayOS và API xác nhận thanh toán thủ công. Việc triển khai webhook giúp hệ thống có thể cập nhật trạng thái thanh toán theo thời gian thực ngay sau khi giao dịch hoàn tất.

Đối với quản trị viên, Payment Service hỗ trợ các API quản lý phương thức thanh toán như thêm mới, cập nhật hoặc vô hiệu hóa phương thức thanh toán. Ngoài ra, service còn cung cấp Internal API nhằm hỗ trợ Order Service tạo giao dịch thanh toán hoặc sinh liên kết thanh toán trực tuyến trong quá trình xử lý đơn hàng.

Thiết kế này giúp Payment Service hoạt động độc lập với Order Service, đồng thời vẫn đảm bảo khả năng phối hợp xử lý nghiệp vụ thanh toán trong hệ thống phân tán.

2.6.7. Thiết kế API của Recommendation Service

Recommendation Service cung cấp các API phục vụ chức năng gợi ý sản phẩm và cá nhân hóa trải nghiệm người dùng.

Service này hỗ trợ các API đề xuất sản phẩm tương đồng dựa trên collaborative filtering cũng như các API gợi ý sản phẩm theo hành vi tương tác của người dùng. Ngoài ra, hệ thống còn hỗ trợ API dành cho quản trị viên nhằm thực hiện tính toán lại dữ liệu recommendation khi dữ liệu sản phẩm hoặc hành vi người dùng có sự thay đổi lớn.

Việc tách riêng Recommendation Service giúp hệ thống dễ dàng mở rộng các thuật toán phân tích dữ liệu hoặc tích hợp các mô hình Machine Learning trong tương lai mà không ảnh hưởng đến các service nghiệp vụ chính.

2.6.8. Thiết kế chuẩn Request và Response

Toàn bộ API trong hệ thống sử dụng chung một chuẩn response nhằm đảm bảo tính nhất quán trong quá trình giao tiếp giữa frontend và backend. Các response được đóng gói dưới dạng ApiResponse<T> với các thành phần như mã trạng thái, thông báo xử lý và dữ liệu trả về.

Việc chuẩn hóa response giúp frontend dễ dàng xử lý dữ liệu và giảm sự khác biệt giữa các service trong hệ thống. Ngoài ra, các request sau khi được xác thực thành công tại API Gateway sẽ được bổ sung thêm các header như X-User-Id, X-User-Email và X-User-Roles trước khi chuyển tiếp đến service xử lý.

Các service phía sau sẽ đọc trực tiếp thông tin người dùng từ header thay vì thực hiện xác thực JWT lại. Cách tiếp cận này giúp giảm tải xử lý bảo mật cho từng service, đồng thời nâng cao hiệu năng và đảm bảo tính tập trung trong cơ chế xác thực của hệ thống.

2.7. Thiết kế Message Queue

Trong kiến trúc Microservice, việc các service giao tiếp trực tiếp với nhau thông qua RESTful API có thể làm tăng mức độ phụ thuộc giữa các thành phần trong hệ thống. Khi một service gặp sự cố hoặc phản hồi chậm, các service liên quan cũng có thể bị ảnh hưởng theo. Để giảm coupling giữa các service và hỗ trợ xử lý bất đồng bộ, hệ thống sử dụng RabbitMQ làm Message Queue trung tâm cho cơ chế truyền nhận event giữa các microservice.

RabbitMQ được sử dụng chủ yếu cho các nghiệp vụ không yêu cầu phản hồi tức thời như gửi email thông báo, cập nhật tồn kho, cập nhật recommendation hoặc xử lý trạng thái thanh toán. Thay vì để các service gọi trực tiếp lẫn nhau trong cùng request, hệ thống sử dụng mô hình Event-Driven Architecture, trong đó service phát sinh nghiệp vụ sẽ publish event lên Message Queue và các service liên quan sẽ tiếp nhận event để xử lý độc lập. Cách tiếp cận này giúp tăng khả năng mở rộng, nâng cao tính chịu lỗi và hạn chế sự phụ thuộc trực tiếp giữa các service trong hệ thống phân tán.

Hệ thống sử dụng các topic exchange chính bao gồm:

- neki.order.exchange cho các sự kiện liên quan đến đơn hàng
- neki.payment.exchange cho các sự kiện liên quan đến thanh toán
- neki.user.exchange cho các sự kiện liên quan đến tài khoản người dùng

Việc phân chia exchange theo từng miền nghiệp vụ giúp hệ thống dễ dàng quản lý luồng event, đồng thời hỗ trợ mở rộng thêm các service xử lý mới trong tương lai mà không làm thay đổi cấu trúc hiện tại của hệ thống.

2.7.1. Các queue sử dụng

Các queue được thiết kế theo hướng tách riêng theo service, giúp mỗi service có thể xử lý cùng một event theo nhu cầu riêng mà không tranh chấp message với service khác.

Bảng 2.1. Thiết kế các queue sử dụng.

Queue	Service	Mục đích
notification.order.created	Notification Service	Gửi email xác nhận đơn hàng
product.order.created	Product Service	Xác nhận tồn kho đã giữ cho đơn hàng
recommendation.order.created.queue	Recommendation Service	Xóa cache gợi ý của người dùng sau khi mua hàng
notification.order.cancelled	Notification Service	Gửi email thông báo hủy đơn
product.order.cancelled	Product Service	Khôi phục tồn kho khi đơn hàng bị hủy
notification.order.updated	Notification Service	Gửi email khi trạng thái đơn hàng thay đổi
order.payment.completed	Order Service	Cập nhật đơn hàng sang trạng thái CONFIRMED

notification.payment	Notification Service	Gửi email xác nhận thanh toán
order.payment.failed	Order Service	Queue đã khai báo cho luồng thanh toán thất bại
notification.user.registered	Notification Service	Gửi OTP xác thực hoặc email chào mừng
notification.user.password	Notification Service	Gửi OTP đặt lại mật khẩu

Việc tổ chức queue theo từng nghiệp vụ giúp hệ thống dễ dàng mở rộng số lượng consumer, đồng thời cho phép nhiều service xử lý song song cùng một event mà không ảnh hưởng lẫn nhau.

2.7.2. *Producer và Consumer*

Các service trong hệ thống có thể đóng vai trò producer hoặc consumer tùy từng nghiệp vụ:

- User Service đóng vai trò producer cho các event user.registered và user.forgot.password. Notification Service là consumer và xử lý việc gửi email OTP, email chào mừng hoặc email đặt lại mật khẩu.
- Order Service đóng vai trò producer cho các event order.created, order.cancelled và order.status.updated. Các event này được ghi vào bảng outbox trước, sau đó job OutboxEventRelay chạy định kỳ để publish message lên RabbitMQ.
- Product Service là consumer của order.created và order.cancelled, dùng để xác nhận hoặc khôi phục tồn kho.
- Recommendation Service là consumer của order.created, dùng để xóa cache recommendation của người dùng có hành vi mua hàng mới.
- Payment Service đóng vai trò producer cho payment.completed và payment.failed sau khi xử lý kết quả thanh toán.

- Order Service là consumer của payment.completed để cập nhật trạng thái đơn hàng. Trong mã nguồn hiện tại, listener cho payment.failed chưa được triển khai đầy đủ nên luồng rollback thanh toán thất bại được ghi nhận là hạn chế của bản demo.
- Notification Service là consumer của nhiều nhóm event để tách tác vụ gửi email khỏi luồng request chính.

Thiết kế producer-consumer theo hướng phân tán giúp hệ thống giảm phụ thuộc trực tiếp giữa các service, đồng thời hỗ trợ mở rộng độc lập từng thành phần xử lý event theo nhu cầu thực tế.

2.7.3. *Luồng message*

Hệ thống sử dụng cơ chế giao tiếp bất đồng bộ dựa trên event nhằm hỗ trợ xử lý các nghiệp vụ liên quan đến đơn hàng, thanh toán và thông báo người dùng. Luồng message quan trọng nhất của hệ thống là luồng đặt hàng và thanh toán.

Step 1: Người dùng gửi request tạo đơn hàng đến API Gateway.

Step 2: API Gateway xác thực JWT và chuyển request đến Order Service.

Step 3: Order Service kiểm tra thông tin người dùng, sản phẩm và tồn kho thông qua Internal API/Feign Client.

Step 4: Sau khi tạo đơn hàng thành công, Order Service lưu event vào bảng outbox.

Step 5: Job OutboxEventRelay đọc event còn trạng thái PENDING, publish message lên RabbitMQ và chuyển event sang trạng thái PROCESSED nếu publish thành công.

Step 6: Notification Service nhận order.created để gửi email xác nhận đơn hàng.

Step 7: Product Service nhận order.created để xác nhận tồn kho đã giữ.

Step 8: Recommendation Service nhận order.created để xóa cache gợi ý cá nhân hóa của người dùng.

Step 9: Khi thanh toán hoàn tất, Payment Service publish payment.completed lên RabbitMQ.

Step 10: Order Service nhận `payment.completed` và cập nhật đơn hàng sang trạng thái `CONFIRMED`; Notification Service nhận cùng event để gửi email xác nhận thanh toán.

Trong trường hợp đơn hàng bị hủy hoặc trạng thái đơn hàng thay đổi, Order Service tiếp tục publish các event tương ứng để các service liên quan xử lý các nghiệp vụ tiếp theo như khôi phục tồn kho hoặc gửi thông báo trạng thái đơn hàng.

Thiết kế luồng message theo hướng event-driven giúp hệ thống có khả năng xử lý song song nhiều nghiệp vụ khác nhau mà không làm tăng sự phụ thuộc trực tiếp giữa các service. Ngoài ra, việc sử dụng Message Queue còn giúp hệ thống duy trì khả năng hoạt động ổn định khi một service tạm thời gặp sự cố, do các message vẫn được lưu trữ trong queue và tiếp tục xử lý khi service hoạt động trở lại.

Trong phạm vi dự án, hệ thống tập trung triển khai các cơ chế giao tiếp bất đồng bộ cơ bản nhằm chứng minh khả năng giảm coupling giữa các service và nâng cao khả năng mở rộng của kiến trúc Microservice.

2.8. Thiết kế xử lý lỗi

Trong kiến trúc Microservice, việc xử lý lỗi đóng vai trò quan trọng nhằm đảm bảo tính ổn định, khả năng giám sát và trải nghiệm người dùng trong quá trình vận hành hệ thống. Do hệ thống được phân tán thành nhiều service độc lập, lỗi có thể phát sinh tại nhiều tầng khác nhau như Gateway, service nghiệp vụ, giao tiếp nội bộ hoặc Message Queue. Vì vậy, hệ thống được thiết kế cơ chế xử lý lỗi theo nhiều lớp nhằm chuẩn hóa phản hồi lỗi, giảm ảnh hưởng giữa các service và hỗ trợ theo dõi sự cố trong môi trường thực tế.

Ở tầng API Gateway, hệ thống thực hiện kiểm tra JWT trước khi định tuyến request đến các service phía sau. Các request không hợp lệ như thiếu token, token sai định dạng hoặc token hết hạn sẽ bị từ chối ngay tại Gateway và trả về các mã lỗi xác thực tương ứng như 401 Unauthorized hoặc 403 Forbidden. Thiết kế này giúp ngăn chặn các request không hợp lệ từ sớm, đồng thời giảm tải xử lý bảo mật cho các service phía sau.

Sau khi xác thực thành công, API Gateway sẽ bổ sung các header như `X-User-Id`, `X-User-Email` và `X-User-Roles` vào request trước khi chuyển tiếp đến service xử lý. Các service phía sau sử dụng trực tiếp thông tin này thay vì phải tự giải mã JWT lại. Cách

tiếp cận này giúp tập trung cơ chế bảo mật tại Gateway và nâng cao hiệu năng toàn hệ thống.

Ở tầng service, hệ thống sử dụng cơ chế xử lý lỗi tập trung thông qua GlobalExceptionHandler nhằm chuẩn hóa response lỗi giữa các microservice. Các lỗi nghiệp vụ được định nghĩa thông qua lớp ApplicationException và được chuyển đổi thành response thống nhất dưới dạng ApiResponse. Thiết kế này giúp frontend có thể xử lý lỗi đồng nhất giữa nhiều service khác nhau.

Đối với các lỗi validate dữ liệu đầu vào, hệ thống trả về danh sách các field không hợp lệ nhằm hỗ trợ người dùng dễ dàng xác định nguyên nhân lỗi. Trong khi đó, các lỗi liên quan đến cơ sở dữ liệu hoặc lỗi hệ thống không xác định sẽ được ghi log tại service và trả về response lỗi server phù hợp. Việc chuẩn hóa response lỗi giúp tăng tính nhất quán trong giao tiếp giữa frontend và backend, đồng thời hỗ trợ quá trình bảo trì hệ thống trong tương lai.

Đối với giao tiếp nội bộ giữa các service thông qua Feign Client, hệ thống thiết kế cơ chế fallback nhằm hạn chế ảnh hưởng dây chuyền khi một service tạm thời không khả dụng. Khi service đích gặp lỗi hoặc mất kết nối, service gọi có thể nhận response lỗi đã được chuẩn hóa thay vì xảy ra lỗi crash trực tiếp. Thiết kế này giúp nâng cao tính chịu lỗi và đảm bảo hệ thống vẫn có thể duy trì hoạt động ở một mức độ nhất định khi xảy ra sự cố tại một service riêng lẻ.

Trong cơ chế xử lý bất đồng bộ bằng RabbitMQ, các consumer được thiết kế theo hướng xử lý exception và ghi log khi xảy ra lỗi trong quá trình consume message. Ngoài ra, hệ thống áp dụng Outbox Pattern tại Order Service nhằm giảm nguy cơ mất event trong trường hợp dữ liệu đơn hàng đã được lưu thành công nhưng quá trình publish message gặp lỗi tạm thời. Với cách tiếp cận này, event sẽ được lưu trữ tạm thời trong database và tiếp tục được publish lại khi hệ thống hoạt động ổn định.

Bên cạnh cơ chế xử lý lỗi trong nghiệp vụ, hệ thống còn tích hợp các công cụ monitoring và logging nhằm hỗ trợ giám sát trạng thái hoạt động của các service trong thời gian thực. Các service cung cấp actuator health và metric để Prometheus thực hiện thu thập dữ liệu giám sát định kỳ. Grafana được sử dụng để trực quan hóa metric hệ thống thông qua dashboard monitoring. Ngoài ra, Loki và Promtail hỗ trợ thu thập log

tập trung từ các container nhằm phục vụ việc kiểm tra và phân tích lỗi trong môi trường triển khai phân tán.

Hệ thống cũng hỗ trợ thiết kế cơ chế cảnh báo thông qua Email hoặc Telegram khi phát hiện lỗi hoặc tài nguyên vượt ngưỡng cho phép. Điều này giúp quản trị viên có thể nhanh chóng phát hiện và xử lý các sự cố phát sinh trong quá trình vận hành.

Nhìn chung, thiết kế xử lý lỗi của hệ thống tập trung vào việc chuẩn hóa response lỗi, giảm coupling giữa các service và hỗ trợ khả năng giám sát trong môi trường Microservice. Việc kết hợp xử lý lỗi tập trung, fallback nội bộ, Message Queue và monitoring giúp hệ thống nâng cao khả năng chịu lỗi và đảm bảo tính ổn định trong quá trình vận hành thực tế.

CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG

3.1. Công nghệ sử dụng

Hệ thống được triển khai bằng các công nghệ chính sau:

- Backend: Java 21, Spring Boot 3.5.9, Spring Cloud 2025.0.0.
- API Gateway và Service Discovery: Spring Cloud Gateway, Netflix Eureka.
- Communication: RESTful API, OpenFeign và RabbitMQ.
- Authentication/Authorization: JWT với thư viện JWT, mật khẩu được mã hóa bằng bcrypt.
- Database: MySQL 8; PostgreSQL 16.
- Cache và search engine: Redis 8, Elasticsearch 8.18.0.
- Message Queue: RabbitMQ 4 Management.
- Monitoring và logging: Spring Boot Actuator, Micrometer Prometheus, Prometheus, Grafana, Loki, Tempo, Promtail và Node Exporter.
- Deployment: Docker Compose để chạy toàn bộ hệ thống, hệ thống còn có config để chạy bằng K8S
- Infrastructure: GCP, AWS SES, Cloudflare R2, Grafana, Nginx

3.2. Xây dựng các service



CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
c7c2689c845c	grafana/promtail:3.5.0	neki-promtail	"/usr/bin/promtail -"	59 minutes ago	Up 59 minutes	
9ee9ae9a90b	grafana/tempo:2.7.2	neki-tempo	"/tempo -config.file..."	59 minutes ago	Up 59 minutes (healthy)	0.0.0.0:3200->3200/tcp, [::]:3200->3200/tcp, 0.0.0.0:4317-4318->4317-4318/tcp, [::]:4317-4318->4317-4318/tcp
af5dedd1bc62	grafana/loki:3.5.0	neki-loki	"/usr/bin/loki -conf..."	59 minutes ago	Up 59 minutes (healthy)	0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp
6e91d12ab237	grafana/grafana:latest	neki-grafana	"/run.sh"	3 hours ago	Up 3 hours	0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp
21c52111aa1	prom/prometheus:latest	neki-prometheus	"/bin/prometheus --c..."	3 hours ago	Up 3 hours	0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
54d1526317d8	prom/node-exporter:latest	neki-node-exporter	"/bin/node_exporter -"	3 hours ago	Up 3 hours	0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp
c0be8d5fb343	a94e6802a084	neki-node-exporter	"java -jar app.jar"	3 hours ago	Up 3 hours	0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp
454d76f4c639	7d836cc42924	neki-notification-service	"java -jar app.jar"	3 hours ago	Up About an hour	0.0.0.0:8084->8084/tcp, [::]:8084->8084/tcp
3a721c11c1c9	8353dde6f1e7	neki-payment-service	"java -jar app.jar"	3 hours ago	Up About an hour	0.0.0.0:8083->8083/tcp, [::]:8083->8083/tcp
1a437b0e3453	8fffb79e2ece	neki-order-service	"java -jar app.jar"	3 hours ago	Up 3 hours	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
4754cb9d78f	236d6ccba44c	neki-api-gateway	"java -jar app.jar"	3 hours ago	Up 3 hours	0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
6cf850245b3b	4c9b380a8f82	neki-user-service	"java -jar app.jar"	3 hours ago	Up 3 hours	0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp
6cb5ad0becb5	0491fdd91147	neki-product-service	"java -jar app.jar"	3 hours ago	Up 3 hours	0.0.0.0:8085->8085/tcp, [::]:8085->8085/tcp
99ba20084a70	bb476c5e7330	neki-recommendation-service	"java -jar app.jar"	3 hours ago	Up 3 hours (healthy)	0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
81d1c0de272f	mysql:8	neki-service-registry	"docker-entrypoint.s..."	3 hours ago	Up 3 hours (healthy)	33060/tcp, 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
6db2eacbeaf8	elasticsearch:8.18.0	neki-mysql	"/bin/tini -- /usr/L..."	3 hours ago	Up 3 hours (healthy)	0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp, 9306/tcp
528afaf646c4	rabbitmq:4-management-alpine	neki-elasticsearch	"docker-entrypoint.s..."	6 days ago	Up 3 hours (healthy)	4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0
bc5a330b0ab	redis:8.0-alpine	neki-rabbitmq	"docker-entrypoint.s..."	6 days ago	Up 3 hours (healthy)	0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
7161263928a0	postgres:16-alpine	neki-redis	"docker-entrypoint.s..."	8 days ago	Up 3 hours (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

Hình 3.1. Các Docker container của các service trong hệ thống.

Bảng 3.1. Các service và vai trò của nó trong hệ thống.

Thành phần	Port	Vai trò chính	Hạ tầng
Service Registry	8761	Cung cấp Eureka Server để các service đăng ký và phát hiện lẫn nhau	Không phụ thuộc database
API Gateway	8080	Là điểm vào tập trung, định tuyến request, kiểm tra JWT và chuyển tiếp header người dùng	Eureka, Redis cấu hình sẵn cho KeyResolver
User Service	8081	Quản lý đăng ký, đăng nhập, hồ sơ người dùng, OTP và thông tin tài khoản	MySQL, Redis, RabbitMQ
Product Service	8082	Quản lý sản phẩm, danh mục, biến thể, tồn kho, tìm kiếm và hình ảnh	MySQL, Redis, RabbitMQ, Elasticsearch, Cloudinary
Order Service	8083	Quản lý giỏ hàng, đặt hàng, trạng thái đơn hàng và outbox event	MySQL, Redis, RabbitMQ, Feign Client
Payment Service	8084	Quản lý phương thức thanh toán, giao dịch, tích hợp PayOS và webhook/callback	PostgreSQL, RabbitMQ, PayOS

Recommendation Service	8085	Tính toán và cung cấp gợi ý sản phẩm dựa trên tương tác người dùng	PostgreSQL, Redis, RabbitMQ, Feign Client
Notification Service	8086	Xử lý gửi email theo các event nghiệp vụ	RabbitMQ, SMTP

service-registry			
Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - 70af152c5ec2:api-gateway:8080
NOTIFICATION-SERVICE	n/a (1)	(1)	UP (1) - 84ce9d3ed2ca:notification-service:8086
ORDER-SERVICE	n/a (1)	(1)	UP (1) - 73294cbc2494:order-service:8083
PAYMENT-SERVICE	n/a (1)	(1)	UP (1) - e0d7fcb6890d:payment-service:8084
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 82260e3b1df8:product-service:8082
RECOMMENDATION-SERVICE	n/a (1)	(1)	UP (1) - e778162446f4:recommendation-service:8085
USER-SERVICE	n/a (1)	(1)	UP (1) - d0d9a72ab9a7:user-service:8081
General Info			

Hình 3.2. Các service được register vào Eureka để giao tiếp với nhau.

Service Registry được triển khai bằng Netflix Eureka, đóng vai trò là trung tâm đăng ký service. Khi mỗi service khởi động, service đó đăng ký thông tin instance với Eureka. API Gateway và các service có nhu cầu gọi nội bộ có thể sử dụng tên service thay vì cấu hình cứng địa chỉ IP. Điều này phù hợp với môi trường container, nơi địa chỉ IP của container có thể thay đổi sau mỗi lần khởi động lại.

API Gateway là lớp biên của backend. Gateway không xử lý nghiệp vụ thương mại điện tử trực tiếp mà tập trung vào các nhiệm vụ có tính hạ tầng như tiếp nhận request, định tuyến đến service tương ứng, kiểm tra JWT và bổ sung thông tin người dùng vào header trước khi chuyển tiếp. Cách thiết kế này giúp các service phía sau không cần lặp lại toàn bộ logic xác thực token, đồng thời tạo ra một điểm kiểm soát tập trung cho các request đi vào hệ thống.

User Service chịu trách nhiệm quản lý vòng đời tài khoản người dùng. Service này lưu dữ liệu chính trong MySQL, sử dụng Redis cho một số dữ liệu tạm thời hoặc dữ liệu cần truy cập nhanh, đồng thời phát sinh event qua RabbitMQ cho các nghiệp vụ như đăng ký tài khoản hoặc quên mật khẩu. Khi người dùng thực hiện các thao tác cần gửi

email OTP, việc gửi email không được xử lý trực tiếp trong request chính mà được chuyển sang Notification Service thông qua message queue.

Product Service quản lý dữ liệu sản phẩm, danh mục, biến thể sản phẩm và tồn kho. Dữ liệu nghiệp vụ chính được lưu trong MySQL, trong khi Elasticsearch được sử dụng để hỗ trợ các tác vụ tìm kiếm và lọc sản phẩm. Service này cũng kết nối Cloudinary để phục vụ nhu cầu lưu trữ hình ảnh sản phẩm. Trong các luồng liên quan đến đơn hàng, Product Service không chủ động xử lý toàn bộ nghiệp vụ đặt hàng mà nhận event từ Order Service để xác nhận hoặc khôi phục tồn kho.

Order Service là service trung tâm của luồng mua hàng. Service này phối hợp với User Service để xác định thông tin người dùng, với Product Service để kiểm tra sản phẩm và tồn kho, và với Payment Service để tạo bản ghi thanh toán hoặc liên kết thanh toán. Do luồng đặt hàng có nhiều bước và liên quan đến nhiều service, Order Service sử dụng thêm outbox để lưu event trước khi publish lên RabbitMQ. Cách tiếp cận này giúp giảm nguy cơ mất event khi nghiệp vụ tạo đơn hàng đã thành công nhưng việc gửi message gặp lỗi tạm thời.

Payment Service được tách riêng nhằm cô lập các nghiệp vụ liên quan đến giao dịch thanh toán. Service này sử dụng PostgreSQL để lưu thông tin phương thức thanh toán và bản ghi giao dịch. Khi thanh toán qua PayOS hoàn tất, Payment Service cập nhật trạng thái trong cơ sở dữ liệu riêng và publish event kết quả thanh toán để các service khác xử lý tiếp. Việc tách Payment Service khỏi Order Service giúp mô hình hệ thống rõ ràng hơn, đồng thời hạn chế việc các service khác truy cập trực tiếp vào dữ liệu thanh toán.

Recommendation Service đảm nhận chức năng gợi ý sản phẩm. Service này lấy dữ liệu tương tác từ Order Service và thông tin sản phẩm từ Product Service thông qua Feign Client, sau đó lưu dữ liệu phân tích vào PostgreSQL và cache kết quả bằng Redis. Trong bản demo hiện tại, Recommendation Service sử dụng hướng tiếp cận dựa trên dữ liệu tương tác và độ tương đồng sản phẩm, phù hợp để minh họa việc một microservice có thể khai thác dữ liệu từ nhiều service khác nhưng vẫn giữ vùng lưu trữ riêng.

Notification Service được thiết kế theo hướng event-driven. Service này không được gọi trực tiếp trong các request chính của người dùng mà lắng nghe các event từ RabbitMQ, sau đó gửi email thông báo tương ứng. Cách tách riêng này giúp các nghiệp

vụ như đăng ký tài khoản, tạo đơn hàng, hủy đơn hàng, cập nhật trạng thái đơn hàng hoặc thanh toán thành công không bị phụ thuộc trực tiếp vào tốc độ gửi email.

3.3. Xây dựng giao tiếp RESTful API

Giao tiếp đồng bộ trong hệ thống được thiết kế theo phong cách RESTful API. API Gateway là điểm vào chính của toàn bộ backend, nhận request từ frontend và định tuyến đến các service nghiệp vụ dựa trên đường dẫn URL. Việc gom điểm vào tại Gateway giúp frontend không cần biết chi tiết vị trí triển khai của từng service phía sau.

Ở lớp định tuyến, API Gateway sử dụng cơ chế service discovery thông qua Eureka. Thay vì chuyển tiếp request đến một địa chỉ cụ thể, Gateway định tuyến theo dạng `lb://service-name`. Cơ chế này giúp hệ thống giảm phụ thuộc vào địa chỉ vật lý của từng instance. Trong môi trường Docker Compose hiện tại, mỗi service thường có một instance, nhưng thiết kế này vẫn tạo tiền đề cho việc mở rộng số lượng instance trong tương lai.

Định dạng dữ liệu trao đổi giữa client và backend chủ yếu là JSON. Các response nghiệp vụ được chuẩn hóa bằng cấu trúc `ApiResponse<T>`, bao gồm các trường như thời điểm phản hồi, mã trạng thái, trạng thái xử lý, thông báo, dữ liệu trả về và danh sách lỗi nếu có. Việc chuẩn hóa response có ý nghĩa quan trọng trong hệ thống nhiều service, vì nếu mỗi service trả response theo một kiểu khác nhau thì frontend sẽ phải xử lý nhiều trường hợp đặc biệt, làm tăng độ phức tạp của giao diện người dùng.

Đối với các API yêu cầu xác thực, Gateway kiểm tra JWT trước khi định tuyến. Khi token hợp lệ, Gateway bổ sung các header như `X-User-Id`, `X-User-Email` và `X-User-Roles` vào request. Các service phía sau sử dụng thông tin này để xác định người dùng hiện tại hoặc kiểm tra quyền nghiệp vụ mà không cần tự giải mã JWT lại. Cách tổ chức này giúp logic xác thực được tập trung, đồng thời giảm sự lặp lại giữa các service.

Giao tiếp đồng bộ giữa các service được triển khai bằng OpenFeign. Đây là công cụ giúp việc gọi HTTP giữa các service có cú pháp gần giống như gọi một interface nội bộ trong Java. Trong hệ thống, Order Service gọi Product Service để lấy thông tin sản phẩm và xử lý tồn kho, gọi User Service để lấy thông tin người dùng, đồng thời gọi Payment Service để tạo bản ghi hoặc liên kết thanh toán. Recommendation Service gọi Order Service để lấy dữ liệu tương tác và gọi Product Service để lấy thông tin sản phẩm phục vụ gợi ý.

Hệ thống phân biệt giữa API public và API nội bộ. Các endpoint dạng /internal/** được thiết kế để phục vụ giao tiếp giữa các service, không phải API dành cho người dùng cuối. Việc tách nhóm endpoint nội bộ giúp cấu trúc API rõ ràng hơn và giảm khả năng nhầm lẫn giữa các chức năng được phép gọi từ frontend với các chức năng chỉ dành cho service trong hệ thống.

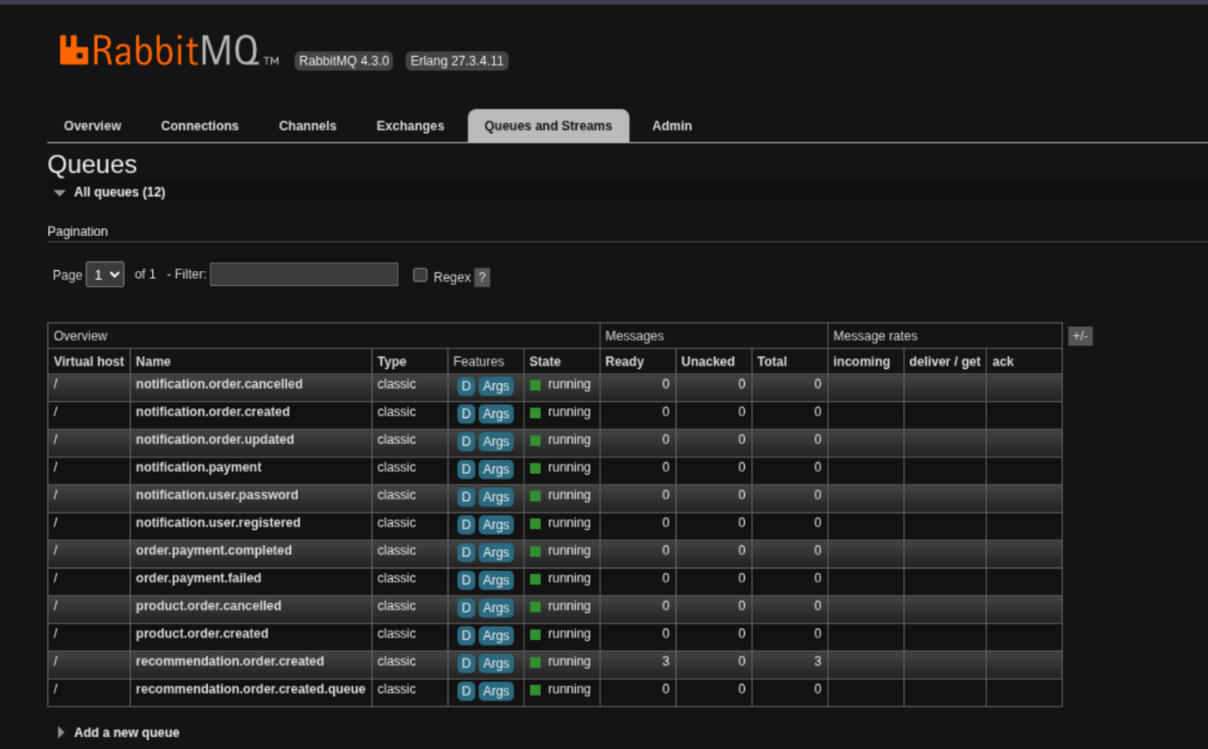
Có thể khái quát các dạng giao tiếp REST chính trong hệ thống như sau:

Bảng 3.2. Các dạng giao tiếp REST chính.

Nhóm giao tiếp	Ví dụ	Mục đích
Client đến Gateway	Frontend gọi /api/v1/products	Cung cấp API cho người dùng cuối
Gateway đến service	Gateway chuyển tiếp đến User/ Product/ Order/ Payment/ Recommendation Service	Định tuyến tập trung và ẩn chi tiết triển khai
Service đến service	Order Service gọi Product/ User/ Payment qua Feign	Hoàn thành nghiệp vụ cần dữ liệu từ service khác
Internal API	Các endpoint / internal/	Phục vụ giao tiếp nội bộ giữa microservice

RESTful API trong hệ thống không chỉ đóng vai trò cung cấp endpoint cho frontend, mà còn là cơ chế giao tiếp đồng bộ giữa các service. Tuy nhiên, không phải mọi tác vụ đều được xử lý bằng REST. Với các nghiệp vụ không cần phản hồi tức thời như gửi email, cập nhật cache gợi ý hoặc xử lý thông báo sau giao dịch, hệ thống sử dụng RabbitMQ để giảm coupling và tăng khả năng chịu lỗi.

3.4. Xây dựng Message Queue



Overview					Messages			Message rates		
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	Incoming	deliver / get	ack
/	notification.order.cancelled	classic	D Args	running	0	0	0			
/	notification.order.created	classic	D Args	running	0	0	0			
/	notification.order.updated	classic	D Args	running	0	0	0			
/	notification.payment	classic	D Args	running	0	0	0			
/	notification.user.password	classic	D Args	running	0	0	0			
/	notification.user.registered	classic	D Args	running	0	0	0			
/	order.payment.completed	classic	D Args	running	0	0	0			
/	order.payment.failed	classic	D Args	running	0	0	0			
/	product.order.cancelled	classic	D Args	running	0	0	0			
/	product.order.created	classic	D Args	running	0	0	0			
/	recommendation.order.created	classic	D Args	running	3	0	3			
/	recommendation.order.created.queue	classic	D Args	running	0	0	0			

Hình 3.3. Các queue của hệ thống.

Trong một hệ thống Microservice, nếu tất cả các service đều giao tiếp đồng bộ với nhau, một request nghiệp vụ có thể bị kéo dài do phải chờ nhiều service phụ trợ hoàn tất. Điều này làm tăng độ trễ và khiến hệ thống dễ bị ảnh hưởng dây chuyền khi một service phụ trợ gặp lỗi. Vì vậy, hệ thống sử dụng RabbitMQ để triển khai các luồng xử lý bất đồng bộ cho những nghiệp vụ không yêu cầu phản hồi ngay lập tức.

RabbitMQ được triển khai bằng docker. Port 5672 được sử dụng cho giao thức AMQP để các service gửi và nhận message, trong khi port 15672 cung cấp giao diện quản trị phục vụ kiểm tra exchange, queue, binding và trạng thái consumer. Các service Spring Boot sử dụng Spring AMQP để khai báo exchange, queue, binding và listener.

Hệ thống sử dụng ba topic exchange chính gồm order, payment và user. Các exchange này phản ánh ba nhóm sự kiện nghiệp vụ quan trọng: sự kiện đơn hàng, sự kiện thanh toán và sự kiện người dùng. Với topic exchange, một event có thể được định tuyến đến nhiều queue khác nhau dựa trên routing key. Điều này cho phép nhiều service cùng phản ứng trước một sự kiện mà không cần producer biết cụ thể có bao nhiêu consumer phía sau.

Luồng đặt hàng là ví dụ tiêu biểu cho việc sử dụng Message Queue. Khi người dùng tạo đơn hàng thành công, Order Service không gọi trực tiếp Notification Service, Product Service và Recommendation Service trong cùng request. Thay vào đó, Order Service ghi event vào bảng outbox. Job OutboxEventRelay chạy định kỳ 5 giây, đọc tối đa 50 event có trạng thái PENDING, deserialize payload và publish message lên RabbitMQ. Nếu publish thành công, event được đánh dấu PROCESSED; nếu publish thất bại, event vẫn được giữ lại để xử lý trong lần chạy sau.

Cách triển khai outbox này không phải là một giải pháp distributed transaction hoàn chỉnh, nhưng có ý nghĩa quan trọng trong phạm vi đề tài. Nó giúp giảm rủi ro mất event trong tình huống nghiệp vụ chính đã ghi dữ liệu thành công nhưng RabbitMQ tạm thời chưa nhận được message. Thay vì publish trực tiếp rồi phụ thuộc hoàn toàn vào trạng thái của broker tại thời điểm request, hệ thống lưu lại event trong database của Order Service và xử lý publish theo cơ chế nền.

Các consumer trong hệ thống được phân tách theo chức năng. Notification Service có nhiều listener độc lập để xử lý email đăng ký tài khoản, quên mật khẩu, tạo đơn hàng, hủy đơn hàng, cập nhật trạng thái đơn hàng và thanh toán thành công. Product Service lắng nghe các event liên quan đến đơn hàng để xác nhận hoặc khôi phục tồn kho. Recommendation Service lắng nghe event tạo đơn hàng để xóa cache gợi ý của người dùng, giúp kết quả gợi ý sau đó có thể phản ánh hành vi mua hàng mới. Payment Service đóng vai trò producer cho các event payment.completed và payment.failed sau khi xử lý kết quả thanh toán.

Việc sử dụng RabbitMQ đem lại ba lợi ích chính. Thứ nhất, nó giảm coupling giữa các service vì producer chỉ cần phát event theo routing key, không cần biết consumer cụ thể. Thứ hai, nó giúp tách các tác vụ chậm như gửi email ra khỏi request chính. Thứ ba, nó hỗ trợ hệ thống hoạt động ổn định hơn khi một service phụ trợ tạm thời chưa sẵn sàng, do message có thể được giữ trong queue cho đến khi consumer xử lý.

3.5. Triển khai Docker Compose

Để triển khai một hệ thống gồm nhiều service và nhiều thành phần hạ tầng, Docker Compose được sử dụng như công cụ điều phối ở mức môi trường demo. Mỗi microservice có Dockerfile riêng để đóng gói ứng dụng Java thành container độc lập. Các thành phần hạ tầng như MySQL, PostgreSQL, Redis, RabbitMQ và Elasticsearch

sử dụng image chính thức, giúp giảm thời gian cấu hình thủ công và tăng tính tái lập của môi trường kiểm thử.

Bảng 3.3. Các tệp Docker Compose và vai trò trong hệ thống.

File	Vai trò
docker-compose.dev.yml	Run toàn bộ backend service từ source local
docker-compose.monitoring.yml	Khởi chạy Prometheus, Grafana và Node Exporter
docker-compose.logging.yml	Khởi chạy Loki, Tempo và Promtail
docker-compose.nobuild.yml	Chạy app thủ công

Trong `docker-compose.dev.yml`, các service hạ tầng được khai báo trước và có healthcheck để xác định trạng thái sẵn sàng. Ví dụ, MySQL, PostgreSQL, Redis, RabbitMQ và Elasticsearch đều có cơ chế kiểm tra riêng trước khi các service nghiệp vụ phụ thuộc vào chúng khởi động. Điều này giúp giảm lỗi khởi động do service ứng dụng cố gắng kết nối đến database hoặc message broker khi các thành phần đó chưa sẵn sàng.

Các service backend được khai báo trong cùng Docker network. Nhờ vậy, chúng có thể giao tiếp với nhau bằng tên service thay vì địa chỉ IP cụ thể, chẳng hạn `mysql`, `postgres`, `redis`, `rabbitmq`, `elasticsearch` và `service-registry`. Đây là một đặc điểm quan trọng của môi trường container, vì địa chỉ IP nội bộ của container có thể thay đổi sau khi container được tạo lại, trong khi tên service trong Docker network vẫn giữ vai trò định danh ổn định.

Volume được sử dụng để lưu trữ dữ liệu persistent cho các thành phần hạ tầng. Nếu container MySQL, PostgreSQL, Redis, RabbitMQ hoặc Elasticsearch bị xóa và tạo lại, dữ liệu vẫn có thể được giữ lại thông qua named volume. Điều này đặc biệt quan trọng khi hệ thống cần kiểm thử nhiều lần mà không muốn mất dữ liệu mẫu hoặc dữ liệu sinh ra trong quá trình chạy ứng dụng.

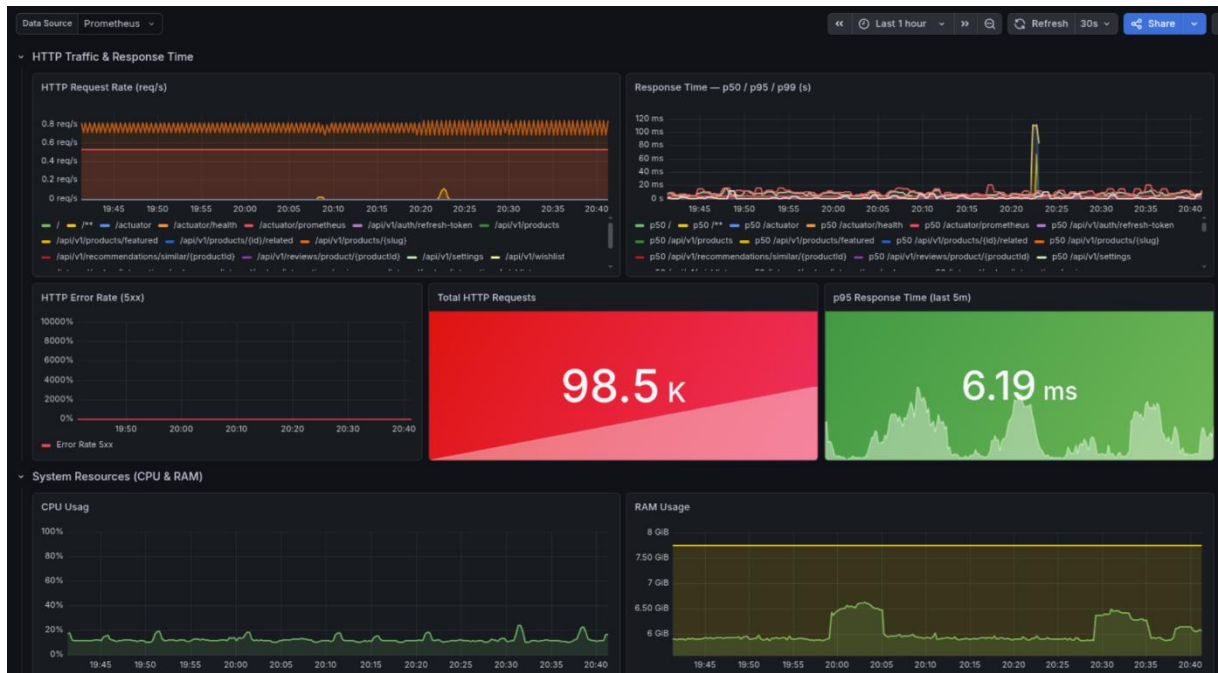
Trong môi trường demo local, một số port được publish ra host để thuận tiện cho việc kiểm tra bằng trình duyệt hoặc công cụ dòng lệnh. API Gateway được expose ở port 8080, Eureka ở 8761, RabbitMQ Management ở 15672, Prometheus ở 9090 và Grafana ở 3001. Việc publish port như vậy phù hợp với môi trường phát triển và kiểm thử, nhưng nếu triển khai production thì các port nội bộ nên được giới hạn sau reverse proxy hoặc firewall. Đặc biệt, các dịch vụ như database, Redis, RabbitMQ broker và Elasticsearch không nên mở công khai nếu không có cơ chế bảo vệ phù hợp.

Docker Compose trong đề tài không nhằm thay thế hoàn toàn các nền tảng orchestration như Kubernetes. Tuy nhiên, với phạm vi demo, Docker Compose đáp ứng tốt yêu cầu khởi chạy nhiều thành phần trong cùng một môi trường, kiểm thử giao tiếp giữa service, giữ dữ liệu bằng volume và quan sát trạng thái container. Đây là lựa chọn phù hợp để chứng minh kiến trúc hệ thống phân tán ở quy mô nhỏ mà không làm tăng quá nhiều chi phí vận hành.

3.6. Logging và Monitoring

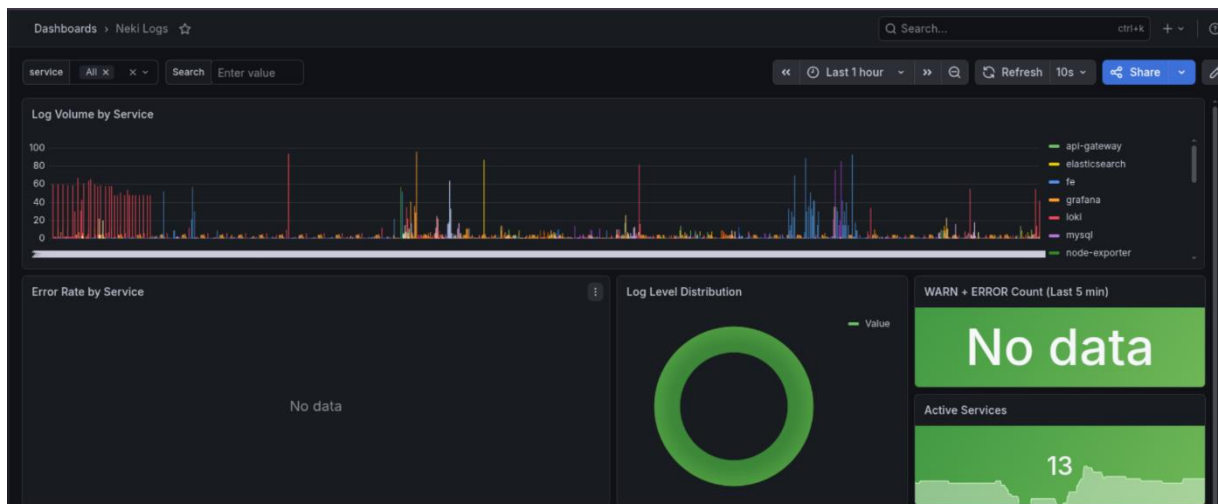
Trong hệ thống phân tán, việc theo dõi trạng thái vận hành có vai trò rất quan trọng vì lỗi có thể xuất hiện ở nhiều điểm khác nhau: API Gateway, service nghiệp vụ, database, cache, message broker hoặc hạ tầng container. Vì vậy, hệ thống được bổ sung các thành phần monitoring và logging nhằm hỗ trợ quan sát, chẩn đoán và đánh giá trạng thái hoạt động.

Mỗi service Spring Boot expose các endpoint actuator như health, metrics và prometheus. Endpoint health giúp xác định service còn hoạt động hay không, trong khi endpoint prometheus cung cấp metric ở định dạng mà Prometheus có thể scrape. Các metric này bao gồm thông tin về JVM, HTTP server, request, lỗi, thời gian xử lý và một số metric hệ thống khác do Spring Boot Actuator và Micrometer cung cấp.



Hình 3.4. Monitoring của server.

Prometheus được cấu hình để scrape các endpoint `/actuator/prometheus` của Service Registry, API Gateway và các service nghiệp vụ. Ngoài metric ứng dụng, Node Exporter được sử dụng để thu thập metric của máy host như CPU, RAM, filesystem và network. Nhờ vậy, việc giám sát không chỉ dừng lại ở trạng thái service mà còn bao gồm tình trạng tài nguyên của môi trường chạy.



Hình 3.5. Logging hệ thống qua Loki + Promtail (1).

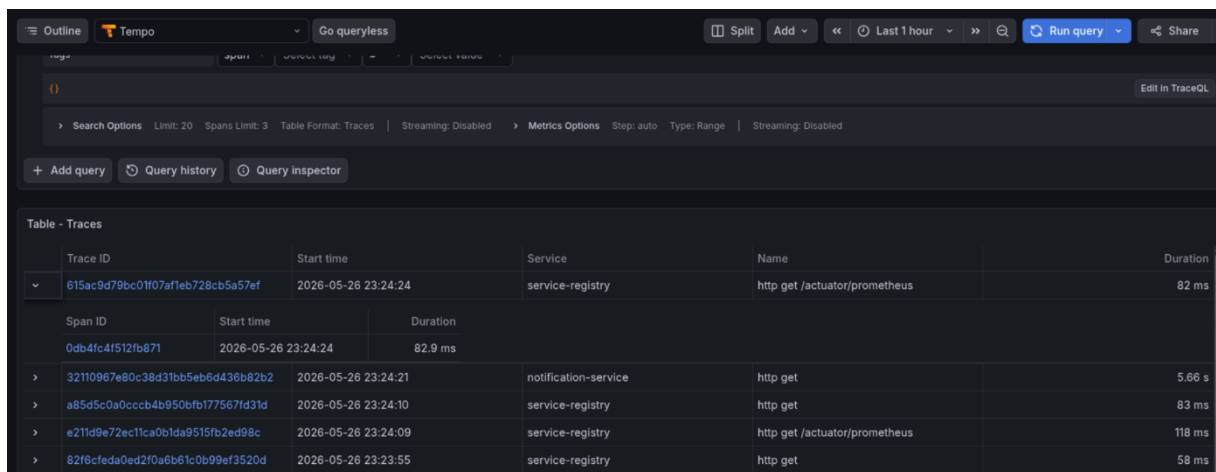
Grafana đóng vai trò là lớp trực quan hóa dữ liệu. Grafana được cấu hình datasource đến Prometheus, Loki và Tempo thông qua provisioning. Prometheus cung cấp metric, Loki phục vụ truy vấn log, còn Tempo phục vụ trace. Các dashboard trong thư mục `monitoring/grafana/dashboards` được dùng để quan sát tổng quan trạng thái hệ

thông, metric backend và log container. Việc dùng provisioning giúp cấu hình datasource và dashboard có thể được tái tạo khi container Grafana được khởi động lại.

```
Live Log Stream
: 2026-05-26 23:40:44.388 INFO container=neki-tempo service=tempo service_name=tempo stream=stderr level=info ts=2026-05-26T16:40:44.296157344Z caller=registry.go:255 tenant=single-tenant
msg="collecting metrics" active_series=1548
: 2026-05-26 23:40:43.725 container=neki-order-service service=order-service service_name=order-service stream=stdout 2026-05-26 16:40:43.718 WARN [main] [/] org.hibernate.orm.deprecation - HHH90
00026: MySQL8Dialect has been deprecated; use org.hibernate.dialect.MySQLDialect instead
: 2026-05-26 23:40:43.624 container=neki-order-service service=order-service service_name=order-service stream=stdout 2026-05-26 16:40:43.612 WARN [main] [/] org.hibernate.orm.deprecation - HHH90
00025: MySQL8Dialect does not need to be specified explicitly using 'hibernate.dialect' (remove the property setting and it will be selected by default)
: 2026-05-26 23:40:43.117 INFO container=neki-notification-ser_ service=notification-service service_name=notification-ser_ stream=stdout 2026-05-26 16:40:43.115 INFO [DiscoveryClient-CacheRefres
hExecutor-%d] [/] c.netflix.discovery.DiscoveryClient - Getting all instance registry info from the eureka server
: 2026-05-26 23:40:43.117 INFO container=neki-notification-ser_ service=notification-service service_name=notification-ser_ stream=stdout 2026-05-26 16:40:43.114 INFO [DiscoveryClient-CacheRefres
hExecutor-%d] [/] c.netflix.discovery.DiscoveryClient - Application version is -1: false
: 2026-05-26 23:40:43.117 INFO container=neki-notification-ser_ service=notification-service service_name=notification-ser_ stream=stdout 2026-05-26 16:40:43.114 INFO [DiscoveryClient-CacheRefres
hExecutor-%d] [/] c.netflix.discovery.DiscoveryClient - Registered Applications size is zero : true
: 2026-05-26 23:40:43.112 INFO container=neki-notification-ser_ service=notification-service service_name=notification-ser_ stream=stdout 2026-05-26 16:40:43.112 INFO [DiscoveryClient-CacheRefres
hExecutor-%d] [/] c.netflix.discovery.DiscoveryClient - Application is null : false
: 2026-05-26 23:40:43.112 INFO container=neki-notification-ser_ service=notification-service service_name=notification-ser_ stream=stdout 2026-05-26 16:40:43.112 INFO [DiscoveryClient-CacheRefres
hExecutor-%d] [/] c.netflix.discovery.DiscoveryClient - Force full registry fetch : false
: 2026-05-26 23:40:43.108 INFO container=neki-notification-ser_ service=notification-service service_name=notification-ser_ stream=stdout 2026-05-26 16:40:43.106 INFO [DiscoveryClient-CacheRefres
hExecutor-%d] [/] c.netflix.discovery.DiscoveryClient - Single vip registry refresh property : null
: 2026-05-26 23:40:43.108 INFO container=neki-notification-ser_ service=notification-service service_name=notification-ser_ stream=stdout 2026-05-26 16:40:43.106 INFO [DiscoveryClient-CacheRefres
```

Hình 3.6. Logging hệ thống qua Loki + Promtail (2).

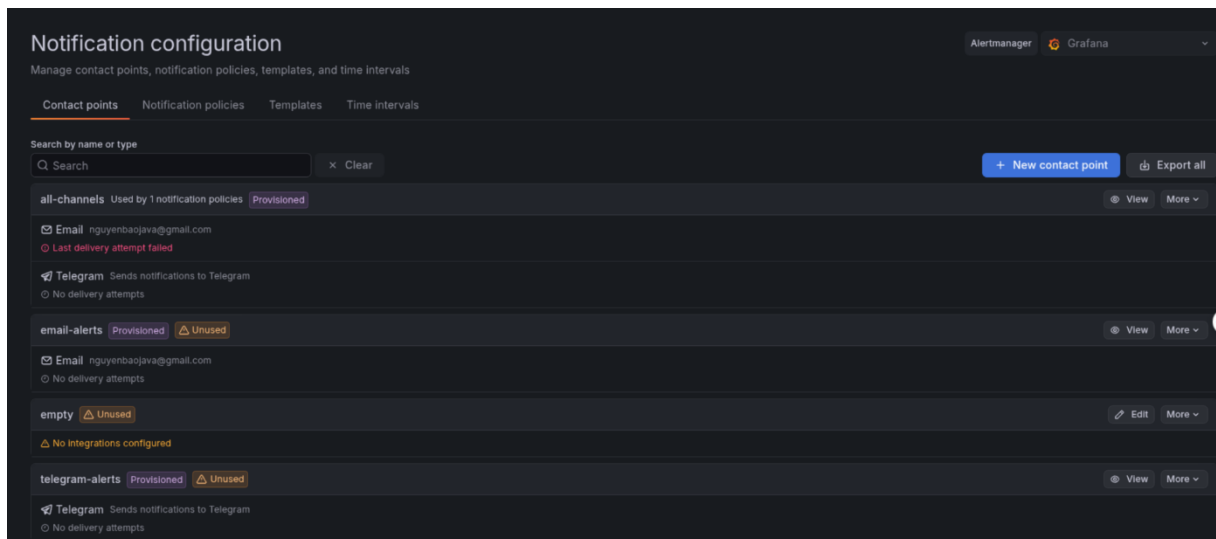
Đối với logging, Promtail đọc log container thông qua Docker socket và gửi về Loki. Khi logging stack được khởi động, người vận hành có thể truy vấn log tập trung trong Grafana thay vì phải truy cập từng container riêng lẻ bằng lệnh docker logs. Điều này có ý nghĩa thực tế trong hệ thống nhiều service, vì một lỗi nghiệp vụ có thể đi qua nhiều thành phần trước khi biểu hiện ra ở frontend.



Trace ID	Start time	Service	Name	Duration
615ac9d79bc01070af1eb728cb5a57ef	2026-05-26 23:24:24	service-registry	http get /actuator/prometheus	82 ms
Span ID	Start time	Duration		
Odb4fc4f512fb871	2026-05-26 23:24:24	82.9 ms		
32110967e80c38d31bb5eb6d436b82b2	2026-05-26 23:24:21	notification-service	http get	5.66 s
a85d5c0a0cccb4b950bf177567fd31d	2026-05-26 23:24:10	service-registry	http get	83 ms
e21fd9e72ec11ca0b1da9515fb2ed98c	2026-05-26 23:24:09	service-registry	http get /actuator/prometheus	118 ms
8216cfeda0ed210a6b61c0b99ef3520d	2026-05-26 23:23:55	service-registry	http get	58 ms

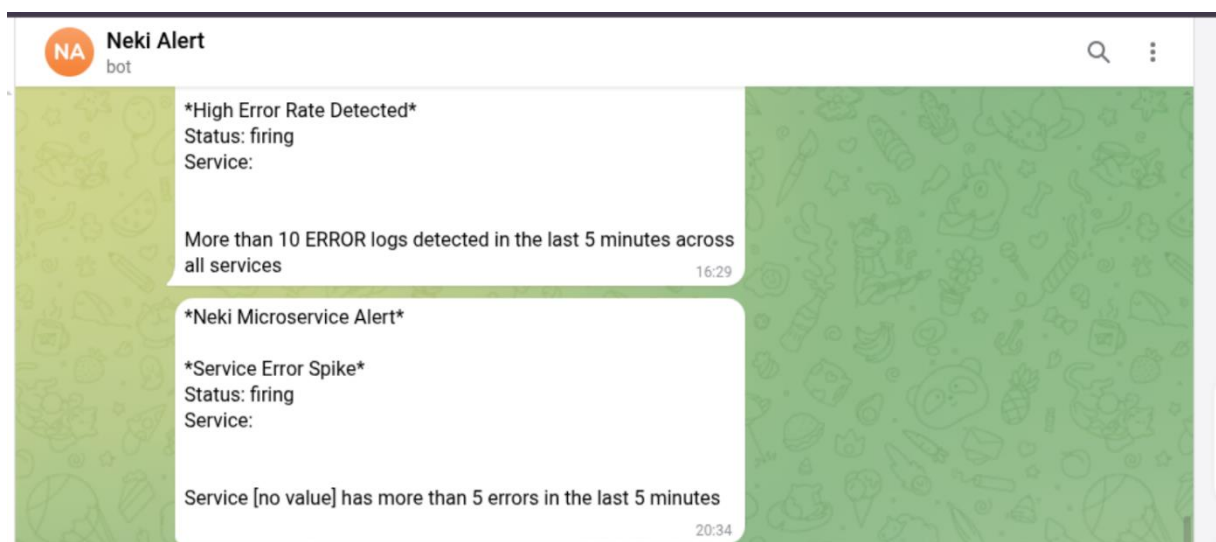
Hình 3.7. Distribute tracing của request tới server.

Tempo được đưa vào hệ thống để chuẩn bị cho nhu cầu quan sát trace phân tán. Trong kiến trúc Microservice, một request có thể đi từ Gateway đến nhiều service phía sau. Trace giúp theo dõi đường đi của request và xác định service nào gây độ trễ hoặc lỗi. Trong phạm vi demo, tracing được cấu hình ở mức hạ tầng quan sát, chưa phải phần được đánh giá sâu như metric và health check.



Hình 3.8. Các cấu hình gửi cảnh báo đến admin.

Hệ thống cũng dùng Grafana để cấu hình alerting với contact point email và Telegram khi có lỗi liên tục hoặc lượng sử dụng RAM và CPU có điểm bất thường.



Hình 3.9. Cảnh báo được gửi qua telegram.

Tại thời điểm kiểm thử local, Prometheus ghi nhận 10 target ở trạng thái up, bao gồm 8 service backend, Prometheus và Node Exporter. Logging stack là thành phần tách riêng nên cần chạy thêm docker-compose.logging.yml nếu muốn thu thập Loki, Tempo và Promtail. Cách chia tách này giúp người triển khai có thể khởi động phần monitoring hoặc logging theo nhu cầu, tránh bắt buộc toàn bộ stack phải chạy trong mọi lần kiểm thử.

CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ

4.1. Môi trường kiểm thử

Trong đề tài này, kiểm thử được thực hiện theo nhiều mức độ khác nhau, trong đó unit test được xem là lớp kiểm thử nền tảng. Mục tiêu của unit test là kiểm tra tính đúng đắn của từng thành phần xử lý nghiệp vụ trong phạm vi nhỏ, trước khi đánh giá toàn bộ hệ thống thông qua các luồng tích hợp giữa các microservice. Cách tiếp cận này phù hợp với kiến trúc Microservice vì mỗi service có ranh giới nghiệp vụ riêng, có thể được kiểm thử độc lập mà không bắt buộc phải khởi động toàn bộ hệ thống Docker Compose.

Môi trường unit test được xây dựng trên nền tảng Java 21, Spring Boot và Maven. Dự án sử dụng spring-boot-starter-test làm bộ thư viện kiểm thử chung cho các module. Thư viện này cung cấp JUnit 5 để định nghĩa và thực thi test case, Mockito để tạo mock cho các dependency, AssertJ/JUnit Assertions để kiểm tra kết quả, cùng một số tiện ích kiểm thử của Spring như mock request, mock exchange và hỗ trợ kiểm thử reactive stream trong API Gateway. Nhờ đó, nhóm có thể kiểm tra logic của service, filter, listener và các lớp tiện ích mà không phụ thuộc trực tiếp vào database, RabbitMQ, Elasticsearch hoặc các service bên ngoài.

Các test case được đặt trong thư mục src/test/java của từng service, đúng với cấu trúc chuẩn của Maven. Toàn bộ hệ thống hiện có 32 lớp kiểm thử, phân bố ở nhiều module như user-service, product-service, order-service, payment-service, notification-service, recommendation-service, api-gateway và common-lib. Việc đặt test trong từng module giúp quá trình kiểm thử phản ánh đúng đặc điểm của microservice: service nào sở hữu logic nào thì service đó cũng chịu trách nhiệm kiểm chứng logic tương ứng.

Trong quá trình viết unit test, các thành phần phụ thuộc được thay thế bằng mock. Ví dụ, khi kiểm thử OrderServiceImpl, các repository, Feign Client gọi sang User Service, Product Service, Payment Service, dịch vụ tính phí vận chuyển và publisher gửi event đều được mock bằng Mockito. Điều này giúp test tập trung vào luồng xử lý của Order Service, chẳng hạn tạo đơn hàng từ giỏ hàng, tạo đơn hàng từ sản phẩm được chọn, mua ngay, kiểm tra quyền truy cập đơn hàng hoặc khôi phục tồn kho khi một bước xử lý thất bại. Tương tự, khi kiểm thử Payment Service, các thao tác với repository và publisher được cô lập để đánh giá riêng logic tạo payment, cập nhật trạng thái thanh toán và phát sinh sự kiện sau thanh toán.

Đối với API Gateway, môi trường kiểm thử sử dụng các đối tượng mock của Spring WebFlux như `MockServerHttpRequest` và `MockServerWebExchange`. Cách này cho phép kiểm tra filter xác thực JWT mà không cần chạy Gateway thật trên port 8080. Các test case có thể mô phỏng request public, request thiếu header Authorization, token sai định dạng, token không hợp lệ, token hợp lệ và request vào đường dẫn yêu cầu quyền admin. Đây là nhóm kiểm thử quan trọng vì Gateway là điểm vào chung của hệ thống, chịu trách nhiệm chặn các request không hợp lệ trước khi chúng đi sâu vào các service nghiệp vụ.

Đối với các service có giao tiếp bất đồng bộ, unit test không khởi động RabbitMQ thật mà mock các lớp gửi và nhận message. Cách làm này giúp kiểm tra nội dung xử lý của listener và publisher ở mức logic: listener có đọc đúng thông tin event hay không, có gọi đúng dịch vụ gửi email hay không, publisher có được kích hoạt sau khi nghiệp vụ hoàn tất hay không. Mặc dù unit test không chứng minh được toàn bộ đường truyền message trong môi trường thật, nó giúp phát hiện sớm lỗi ở phần xử lý event trước khi chuyển sang kiểm thử tích hợp.

Các kiểm thử được thực thi bằng Maven, có thể chạy ở cấp toàn bộ project hoặc theo từng module. Khi phát triển một service cụ thể, nhóm ưu tiên chạy test ở module tương ứng để phản hồi nhanh hơn; sau đó có thể chạy test toàn bộ project nhằm đảm bảo thay đổi ở một module không phá vỡ các phân dùng chung như common-lib, JWT utility hoặc cơ chế exception handling.

4.2. Kiểm thử chức năng

Kiểm thử chức năng ở mức unit test tập trung vào việc xác nhận từng chức năng nghiệp vụ có trả về kết quả đúng trong các tình huống thành công và thất bại thường gặp. Thay vì chỉ kiểm tra API ở bề mặt HTTP, unit test đi sâu hơn vào các lớp service, filter, listener và utility, từ đó giúp nhóm đánh giá logic cốt lõi của hệ thống một cách chủ động hơn. Đây là điểm quan trọng trong hệ thống Microservice, vì một lỗi nhỏ trong từng service có thể lan rộng thành lỗi nghiệp vụ khi các service phối hợp với nhau.

Nhóm test đầu tiên liên quan đến xác thực và phân quyền. Trong user-service, các lớp kiểm thử tập trung vào JWT, lọc request, tải thông tin người dùng và xử lý token. Các test case kiểm tra việc tạo, phân tích và xác thực token; kiểm tra người dùng hợp lệ hoặc không tồn tại; đồng thời đánh giá cách hệ thống phản ứng khi request không mang

thông tin xác thực phù hợp. Ở api-gateway, test của JWT filter kiểm tra rõ sự khác biệt giữa endpoint public và endpoint cần bảo vệ. Các đường dẫn như sản phẩm hoặc đăng nhập được phép đi tiếp mà không cần token, trong khi các đường dẫn như đơn hàng hoặc admin phải có token hợp lệ và quyền phù hợp. Nhóm kiểm thử này giúp đảm bảo cơ chế bảo vệ ở cửa ngõ hệ thống hoạt động đúng trước khi request được chuyển đến các service phía sau.

Nhóm test thứ hai tập trung vào quản lý người dùng và dữ liệu dùng chung. UserServiceImplTest kiểm tra các thao tác nghiệp vụ của người dùng, trong khi SecurityUtilsTest, JwtUtilsTest và CustomUserDetailsServiceTest giúp xác nhận các thành phần hỗ trợ bảo mật hoạt động ổn định. Ở common-lib, các test cho AppException và GlobalExceptionHandler kiểm tra cách hệ thống chuẩn hóa phản hồi lỗi. Điều này có ý nghĩa thực tế vì nhiều service cùng sử dụng thư viện chung; nếu định dạng lỗi được xử lý nhất quán, frontend và các service khác sẽ dễ nhận biết nguyên nhân thất bại hơn.

Nhóm test thứ ba liên quan đến sản phẩm. Các lớp ProductServiceImplTest, CategoryServiceImplTest, AttributeServiceImplTest và BannerServiceImplTest kiểm tra những chức năng chính của Product Service như lấy danh sách sản phẩm, xử lý thông tin danh mục, thuộc tính và banner. Trong các test này, repository được mock để mô phỏng dữ liệu trả về từ tầng lưu trữ. Nhờ đó, test có thể đánh giá riêng logic chuyển đổi dữ liệu, xử lý trường hợp không tìm thấy bản ghi, hoặc kiểm tra dữ liệu đầu ra mà không cần kết nối database thật. Cách kiểm thử này đặc biệt phù hợp với Product Service vì service này có nhiều dữ liệu mô tả sản phẩm và thường được các service khác tham chiếu.

Nhóm test thứ tư là các chức năng đơn hàng. OrderServiceImplTest kiểm tra nhiều tình huống nghiệp vụ quan trọng như tạo đơn hàng từ giỏ hàng, tạo đơn hàng từ danh sách sản phẩm được chọn, mua ngay, lấy chi tiết đơn hàng, kiểm tra quyền truy cập đơn hàng và xử lý lỗi khi dữ liệu không tồn tại. Một điểm đáng chú ý là test có kiểm tra kịch bản bù trừ khi quá trình tạo đơn hàng gặp lỗi sau bước giữ tồn kho. Khi dịch vụ tính phí vận chuyển phát sinh lỗi, Order Service phải gọi lại Product Service để khôi phục tồn kho. Đây là ví dụ thể hiện đặc trưng của hệ thống phân tán: unit test không chỉ kiểm tra kết quả trả về, mà còn kiểm tra hành vi phối hợp và xử lý lỗi giữa các dependency đã được mock.

Bên cạnh đó, các lớp `CartServiceImplTest`, `ShippingServiceImplTest` và `DiscountCalculationServiceImplTest` kiểm tra các phần nhỏ hơn trong nghiệp vụ đặt hàng. `Cart Service` được kiểm thử với các thao tác thêm, cập nhật, xóa sản phẩm trong giỏ hàng; `Shipping Service` kiểm tra logic tính phí vận chuyển; `Discount Calculation Service` kiểm tra cách áp dụng giảm giá vào tổng tiền đơn hàng. Việc tách test theo từng lớp giúp logic nghiệp vụ trở nên dễ quan sát hơn: khi một test thất bại, nhóm có thể nhanh chóng xác định lỗi nằm ở giỏ hàng, phí vận chuyển, giảm giá hay phần tạo đơn hàng chính.

Nhóm test thứ năm liên quan đến thanh toán. Các lớp `PaymentServiceImplTest`, `PaymentMethodServiceImplTest`, `PaymentLinkServiceImplTest` và `PaymentEventPublisherTest` kiểm tra quá trình tạo giao dịch thanh toán, quản lý phương thức thanh toán, tạo liên kết thanh toán và phát sinh event thanh toán. Với một hệ thống thương mại điện tử, thanh toán là chức năng nhạy cảm vì nó ảnh hưởng trực tiếp đến trạng thái đơn hàng và trải nghiệm người dùng. Vì vậy, unit test cần bao phủ cả trường hợp thành công lẫn các tình huống bất thường như không tìm thấy phương thức thanh toán, dữ liệu payment không hợp lệ hoặc sự kiện cần được phát đi sau khi trạng thái thay đổi.

Nhóm test thứ sáu kiểm tra thông báo và giao tiếp bất đồng bộ. Trong `notification-service`, các listener như `UserEventListenerTest`, `OrderEventListenerTest` và `PaymentEventListenerTest` kiểm tra việc tiếp nhận event người dùng, đơn hàng và thanh toán. `EmailServiceImplTest` kiểm tra logic gửi email ở mức service. Các dependency gửi mail hoặc nhận message được mock, do đó test không phụ thuộc vào SMTP server hay RabbitMQ thật. Mục tiêu của nhóm kiểm thử này là đảm bảo khi có event phù hợp, service thông báo sẽ gọi đúng luồng xử lý tương ứng.

Ngoài các nhóm trên, `recommendation-service` cũng có unit test cho logic gợi ý sản phẩm. Chức năng gợi ý thường phụ thuộc vào dữ liệu tương tác, độ tương đồng sản phẩm hoặc lịch sử hành vi người dùng. Trong phạm vi unit test, các nguồn dữ liệu này được giả lập để kiểm tra service có xử lý đúng đầu vào và trả về danh sách gợi ý phù hợp hay không.

Tổng hợp lại, kiểm thử chức năng bằng unit test giúp nhóm kiểm tra các thành phần quan trọng của hệ thống ở mức nhỏ, nhanh và dễ lặp lại. Các test case không thay

thể hoàn toàn kiểm thử tích hợp hoặc kiểm thử end-to-end, nhưng chúng tạo ra lớp bảo vệ đầu tiên cho mã nguồn. Khi một service được thay đổi, unit test giúp phát hiện sớm lỗi logic trước khi hệ thống được chạy trong môi trường Docker và trước khi các luồng nghiệp vụ liên service được kiểm tra ở mức cao hơn.

4.3. Kiểm thử lỗi

Kiểm thử lỗi được thực hiện nhằm đánh giá cách hệ thống phản ứng khi request không hợp lệ, thiếu thông tin xác thực hoặc khi một thành phần xử lý gặp lỗi. Trong hệ thống phân tán, xử lý lỗi có vai trò đặc biệt quan trọng vì lỗi có thể phát sinh ở nhiều lớp khác nhau: Gateway, service nghiệp vụ, database, message broker hoặc service phụ thuộc.

Các tình huống lỗi được kiểm tra và đánh giá gồm:

- Request thiếu JWT đến API yêu cầu xác thực bị API Gateway từ chối với HTTP 401.
- Các API nghiệp vụ sử dụng GlobalExceptionHandler để trả response lỗi theo cấu trúc thống nhất thay vì trả stack trace trực tiếp cho client.
- Nếu request không hợp lệ ở tầng validate DTO, service trả lỗi 400 Bad Request kèm thông tin field lỗi.
- Nếu lỗi SQL hoặc lỗi không xác định xảy ra trong service, lỗi được ghi log và client nhận response lỗi server đã chuẩn hóa.
- Trong luồng outbox, event chưa publish thành công không bị xóa ngay mà vẫn giữ trạng thái PENDING để job xử lý lại.

Cách xử lý lỗi bằng GlobalExceptionHandler giúp response trả về từ các service có định dạng thống nhất. Điều này quan trọng đối với frontend vì giao diện có thể xử lý lỗi dựa trên cùng một cấu trúc thay vì phải phân biệt lỗi từ từng service. Đồng thời, việc không trả stack trace trực tiếp cho client cũng giúp giảm nguy cơ lộ thông tin nội bộ của hệ thống.

Ở lớp Gateway, việc chặn request thiếu JWT cho thấy hệ thống có cơ chế bảo vệ các API yêu cầu xác thực. Các service phía sau không cần tự xử lý mọi trường hợp token không hợp lệ từ đầu, vì request đã được kiểm tra tại lớp biên. Tuy nhiên, các service vẫn

cần kiểm tra các điều kiện nghiệp vụ riêng như quyền truy cập tài nguyên, trạng thái đơn hàng hoặc tính hợp lệ của dữ liệu đầu vào.

Đối với Message Queue, outbox trong Order Service giúp giảm rủi ro mất event khi publish RabbitMQ gặp lỗi tạm thời. Event chưa publish thành công vẫn được giữ trong database để job xử lý lại. Đây là một điểm tích cực trong thiết kế, nhưng vẫn chưa thay thế hoàn toàn cho các cơ chế production-grade như retry giới hạn, Dead Letter Queue và idempotency.

4.4. Đánh giá hệ thống

Từ kết quả thiết kế, triển khai và kiểm thử, có thể đánh giá rằng hệ thống đã đáp ứng được các mục tiêu chính của đề tài ở mức demo. Hệ thống thể hiện được các đặc trưng quan trọng của kiến trúc Microservice như phân rã theo nghiệp vụ, cơ sở dữ liệu riêng theo service, giao tiếp đồng bộ và bất đồng bộ, service discovery, API Gateway, container hóa và giám sát hệ thống.

Các kết quả đạt được gồm:

- Các chức năng nghiệp vụ được tách thành nhiều microservice độc lập.
- API Gateway định tuyến tập trung và kiểm tra JWT trước khi request vào service phía sau.
- Eureka hỗ trợ đăng ký và phát hiện service trong môi trường Docker.
- Dữ liệu được phân tách theo hướng Database per Service với MySQL và PostgreSQL.
- RabbitMQ được dùng cho các luồng bất đồng bộ như order, payment, user và notification.
- Redis, Elasticsearch, Prometheus và Grafana được tích hợp để hỗ trợ cache, tìm kiếm và giám sát.

Về mặt kiến trúc, việc tách User Service, Product Service, Order Service, Payment Service, Recommendation Service và Notification Service giúp hệ thống phản ánh đúng tư duy phân tách trách nhiệm. Mỗi service có vùng dữ liệu riêng và đảm nhận một nhóm nghiệp vụ cụ thể. Cách tổ chức này giúp việc phát triển, kiểm thử và mở rộng từng phần trở nên rõ ràng hơn so với mô hình monolithic.

Về mặt giao tiếp, hệ thống kết hợp RESTful API và RabbitMQ. RESTful API phù hợp với các tác vụ cần phản hồi trực tiếp, trong khi RabbitMQ phù hợp với các tác vụ bất đồng bộ như gửi thông báo, cập nhật tồn kho sau đơn hàng và xóa cache recommendation. Sự kết hợp này giúp hệ thống thể hiện được cả hai mô hình giao tiếp phổ biến trong hệ thống phân tán.

Về mặt triển khai, Docker Compose giúp toàn bộ hệ thống có thể được khởi chạy trong một môi trường nhất quán. Việc có healthcheck, network nội bộ và volume dữ liệu giúp môi trường demo ổn định hơn so với việc chạy từng thành phần thủ công. Bên cạnh đó, monitoring bằng Prometheus và Grafana giúp người vận hành có thể quan sát trạng thái service thay vì chỉ dựa vào log rời rạc.

Nhìn chung, hệ thống chứng minh được cách tổ chức một ứng dụng thương mại điện tử theo kiến trúc Microservice, có giao tiếp đồng bộ qua RESTful API, giao tiếp bất đồng bộ qua RabbitMQ, tách dữ liệu theo service và có nền tảng quan sát hệ thống bằng monitoring/logging. Các hạn chế còn tồn tại không làm mất đi giá trị minh họa của đề tài, nhưng là cơ sở quan trọng để xác định hướng phát triển tiếp theo nếu hệ thống được nâng cấp lên môi trường vận hành thực tế.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Đề tài đã xây dựng được hệ thống thương mại điện tử bán đồ thể thao theo kiến trúc Microservice với các service độc lập như User Service, Product Service, Order Service, Payment Service, Recommendation Service và Notification Service. Hệ thống sử dụng API Gateway làm điểm vào tập trung, Eureka Service Registry để đăng ký và phát hiện service, RESTful API cho giao tiếp đồng bộ và RabbitMQ cho giao tiếp bất đồng bộ giữa các service.

Hệ thống được thiết kế theo nguyên tắc phân tách miền nghiệp vụ, giúp giảm sự phụ thuộc trực tiếp giữa các thành phần và hỗ trợ triển khai độc lập cho từng service. Mỗi service sử dụng cơ sở dữ liệu riêng nhằm đảm bảo nguyên tắc Database per Service. Trong đó, MySQL được sử dụng cho các service nghiệp vụ chính như người dùng, sản phẩm và đơn hàng; PostgreSQL được sử dụng cho thanh toán và gợi ý sản phẩm; Redis hỗ trợ cache dữ liệu và Elasticsearch hỗ trợ tìm kiếm sản phẩm.

Về mặt triển khai, toàn bộ hệ thống được container hóa và vận hành thông qua Docker Compose nhằm đảm bảo tính nhất quán giữa các môi trường phát triển và kiểm thử. Ngoài ra, hệ thống còn tích hợp các công cụ monitoring và centralized logging như Prometheus, Grafana, Loki và Promtail nhằm hỗ trợ theo dõi trạng thái hoạt động của các service trong thời gian thực.

Kết quả kiểm thử cho thấy các service chính hoạt động ổn định, các API giao tiếp thành công thông qua API Gateway, RabbitMQ xử lý được các luồng message giữa các service và hệ thống monitoring có thể thu thập metric cũng như log từ các service đang hoạt động. Những kết quả này cho thấy hệ thống có thể vận hành như một hệ thống Microservice cơ bản với cơ chế giao tiếp và phối hợp giữa nhiều service độc lập.

2. Hạn chế và các vấn đề còn tồn tại

Mặc dù hệ thống đã triển khai được các thành phần cốt lõi của kiến trúc Microservice, đề tài vẫn còn một số hạn chế cần tiếp tục hoàn thiện.

Do hệ thống được xây dựng theo mô hình phân tán nên việc quản lý giao tiếp giữa các service phức tạp hơn so với kiến trúc Monolithic. Việc xử lý lỗi trong giao tiếp bất đồng bộ thông qua RabbitMQ vẫn chủ yếu ở mức cơ bản và chưa triển khai đầy đủ các

cơ chế như Dead Letter Queue, retry policy hoặc idempotency để xử lý message lỗi và tránh xử lý trùng lặp.

Ngoài ra, việc đảm bảo tính nhất quán dữ liệu giữa các service vẫn là một thách thức do mỗi service sử dụng cơ sở dữ liệu riêng biệt. Trong phạm vi đề tài, hệ thống chưa triển khai các mô hình distributed transaction hoặc Saga Pattern để xử lý các giao dịch phân tán phức tạp.

Về mặt triển khai, hệ thống hiện chủ yếu hoạt động trong môi trường Docker Compose phục vụ mục đích phát triển và kiểm thử. Các cơ chế triển khai production như Kubernetes cluster, autoscaling, service mesh hoặc quản lý secret chưa được triển khai thực tế. Bên cạnh đó, hệ thống cũng chưa thực hiện kiểm thử tải hoặc tối ưu hiệu năng ở quy mô lớn.

3. Hướng phát triển

Trong tương lai, hệ thống có thể tiếp tục được mở rộng và hoàn thiện theo các hướng sau:

- Triển khai Kubernetes nhằm hỗ trợ orchestration và autoscaling cho các service.
- Bổ sung các cơ chế xử lý lỗi nâng cao như Dead Letter Queue, retry policy và circuit breaker.
- Áp dụng Saga Pattern để cải thiện xử lý distributed transaction giữa các service.
- Tích hợp CI/CD pipeline nhằm tự động hóa quá trình build và triển khai hệ thống.
- Nâng cấp Recommendation Service bằng các thuật toán Machine Learning để cải thiện chất lượng gợi ý sản phẩm.
- Tăng cường bảo mật hệ thống thông qua quản lý secret, rate limiting và kiểm thử bảo mật.
- Thực hiện kiểm thử tải và tối ưu hiệu năng hệ thống trong môi trường có số lượng lớn người dùng đồng thời.

Việc tiếp tục hoàn thiện các nội dung trên sẽ giúp hệ thống tiệm cận gần hơn với mô hình Microservice được triển khai trong môi trường thực tế.

TÀI LIỆU THAM KHẢO

- [1] [1] Học viện Công nghệ Bưu chính Viễn thông, Giáo trình Các hệ thống phân tán, Hà Nội, 2024.
- [2] Spring Documentation, “Spring Boot Reference Documentation”. [Online]. Available: <https://spring.io/projects/spring-boot>.
- [3] Docker Documentation, “Docker Compose Documentation”. [Online]. Available: <https://docs.docker.com/compose/>.
- [4] RabbitMQ Documentation, “RabbitMQ Messaging Broker”. [Online]. Available: <https://www.rabbitmq.com/>.